DBMS COURSE PROJECT · FINAL PRESENTATION

Hostel Accommodation and Student Services Management System

Screenshots of every screen of the user interface: insert, delete and view, before and after, with the matching MySQL output.

GROUP MEMBERS

Mohammed Amaan Khan · 25WU0101075

Arslan Alam · 25WU0101016

Jaswant Reddy · 25WU0101056

Contents

Application screens 3
1. View: a student's room and requests 3
2. Insert: a new service request (student Sneha Reddy) 5
3. Delete: cancel an open request 8
4. Service workflow: Open, Assigned, In Progress, Resolved 10
5. Room allotment by the warden 13
6. Rent payment 17
7. Insert and delete a record: department 21
8. Database tab: what is stored in MySQL 25

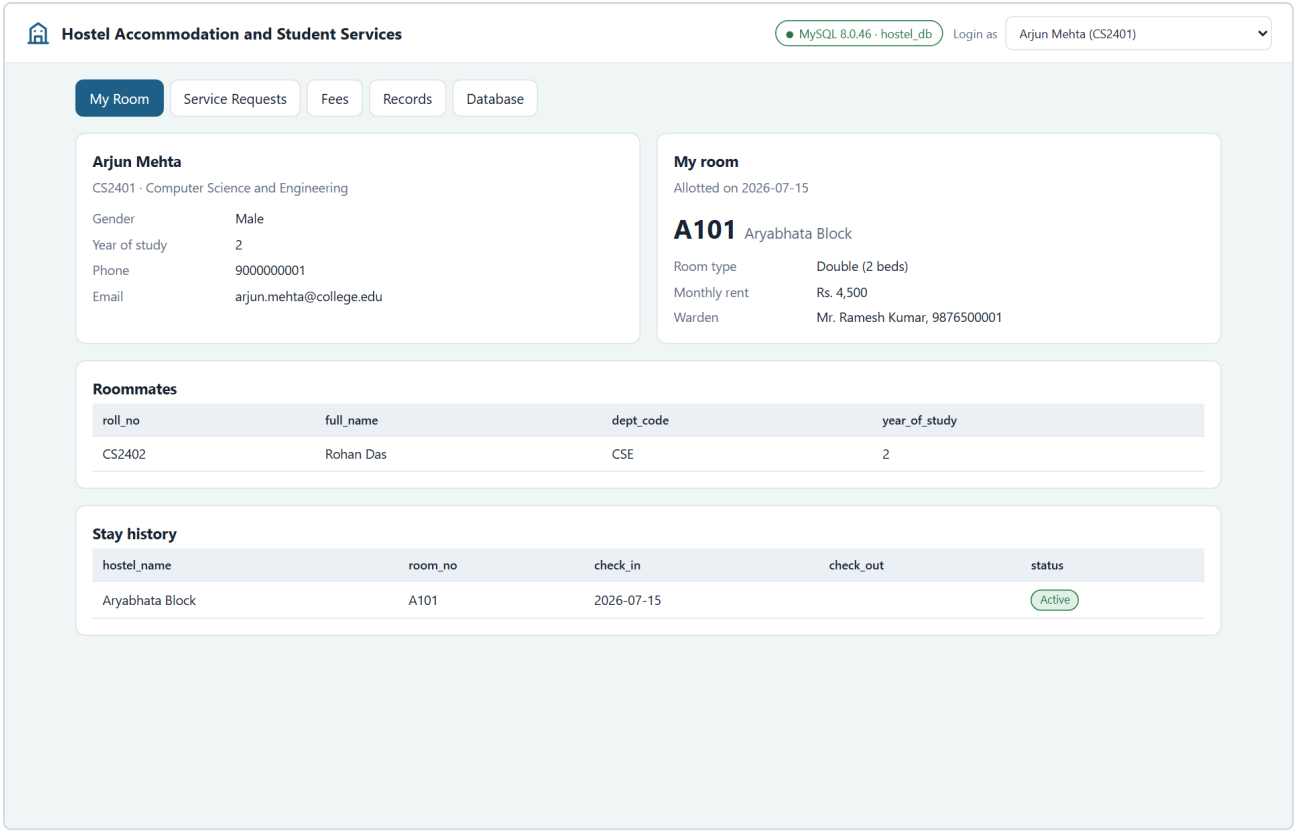
Application screens

Insert, delete and view, each shown before and after. The interface runs on Node.js and reads and writes the MySQL database hostel_db directly.

1. View: a student's room and requests

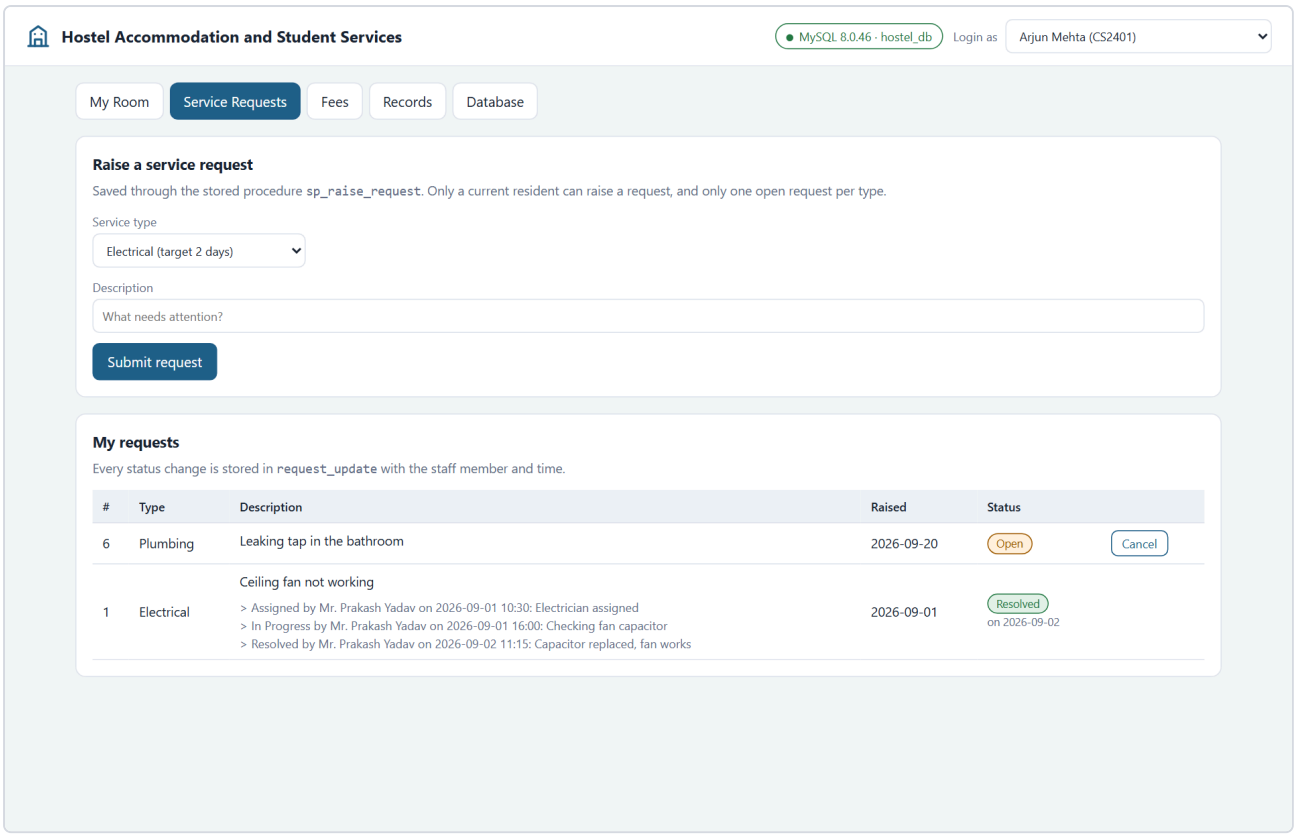
VIEW Figure 1. My Room, student Arjun Mehta

Profile, the room allotted to him, his roommate and his stay history, all read from MySQL.



VIEW **Figure 2. Service Requests, Arjun Mehta**

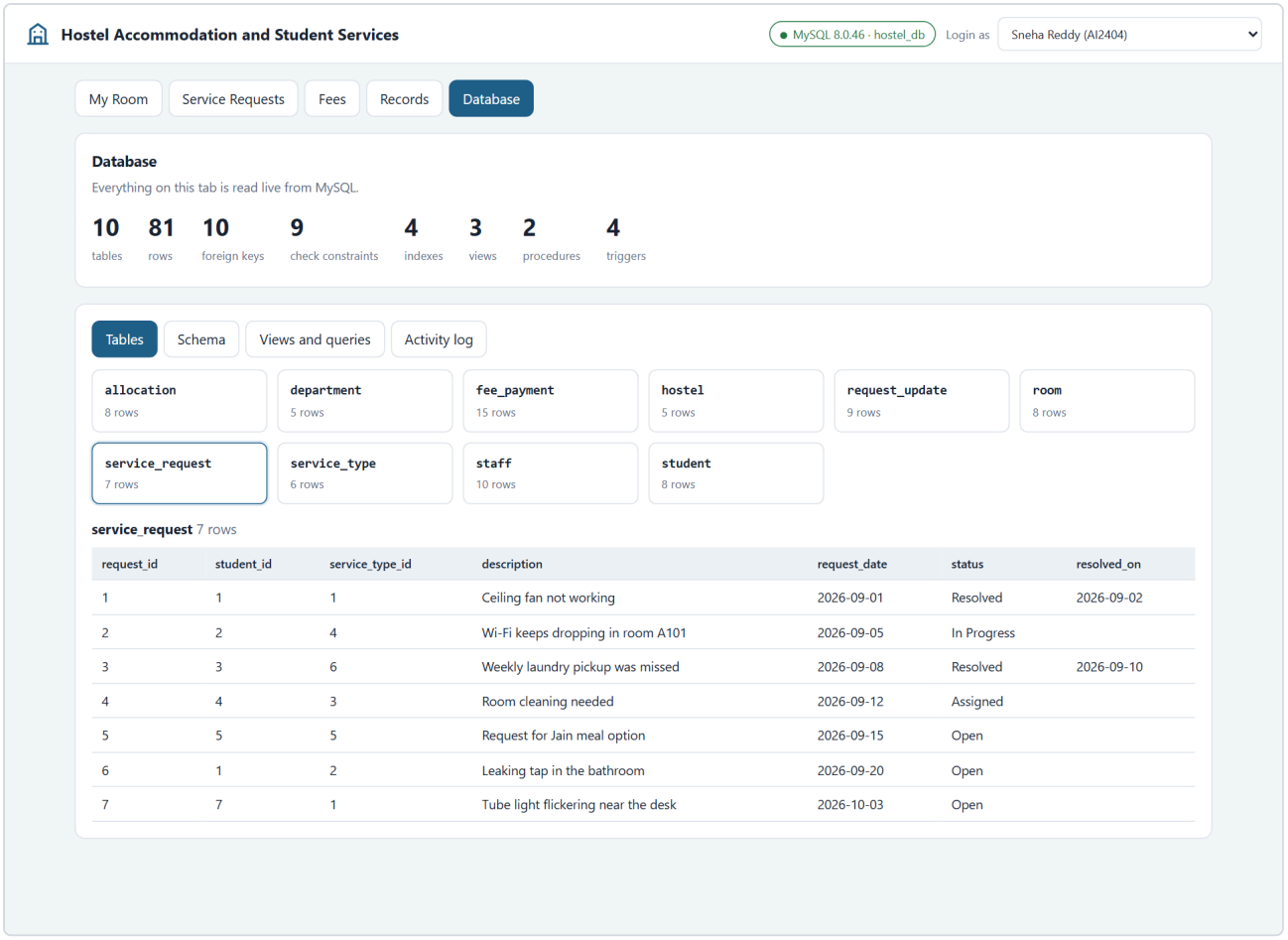
Request #1 is Resolved and lists every step with the staff member and time. Request #6 is still Open.



2. Insert: a new service request (student Sneha Reddy)

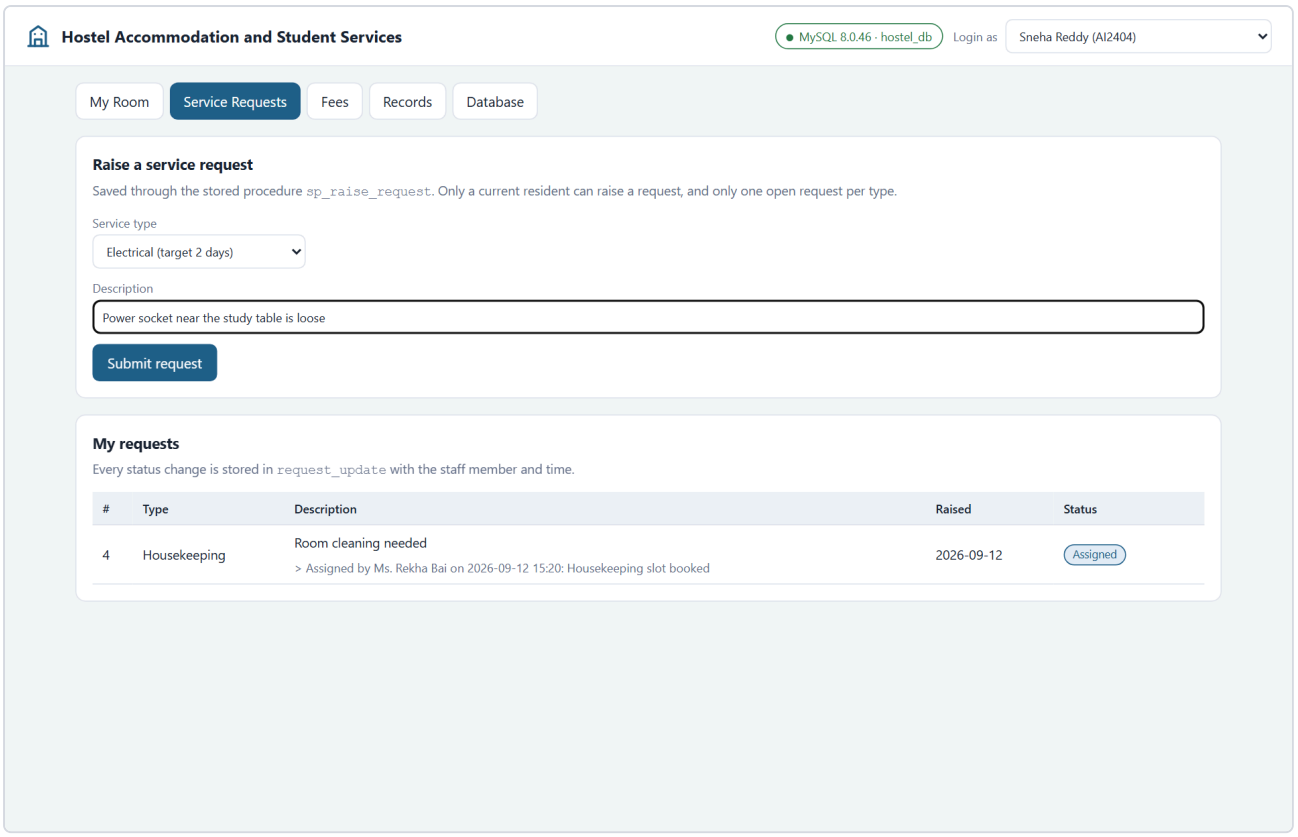
INSERT Figure 3. Before: service_request table in the Database tab

The Database tab reads the table straight from MySQL. service_request holds 7 rows.



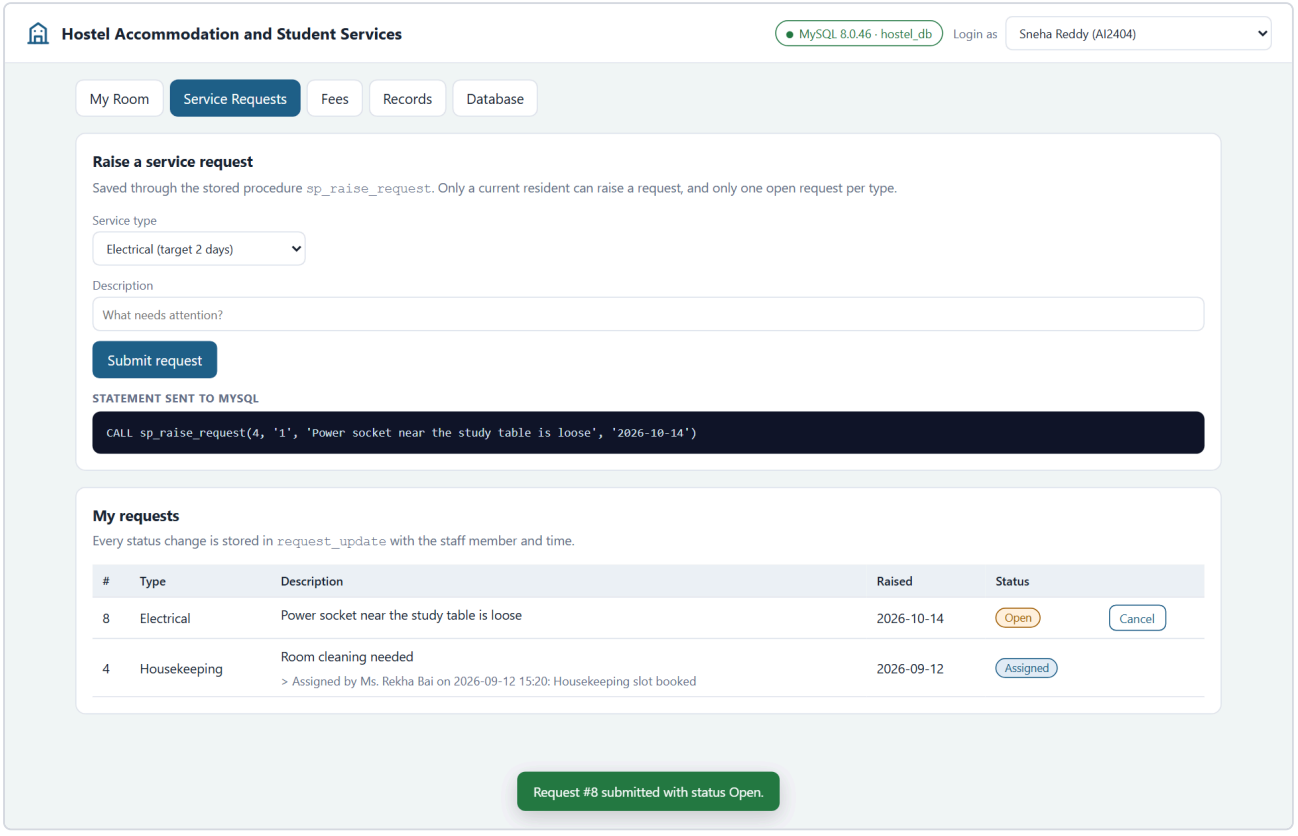
INSERT Figure 4. Service request form filled in

Type Electrical, description "Power socket near the study table is loose".



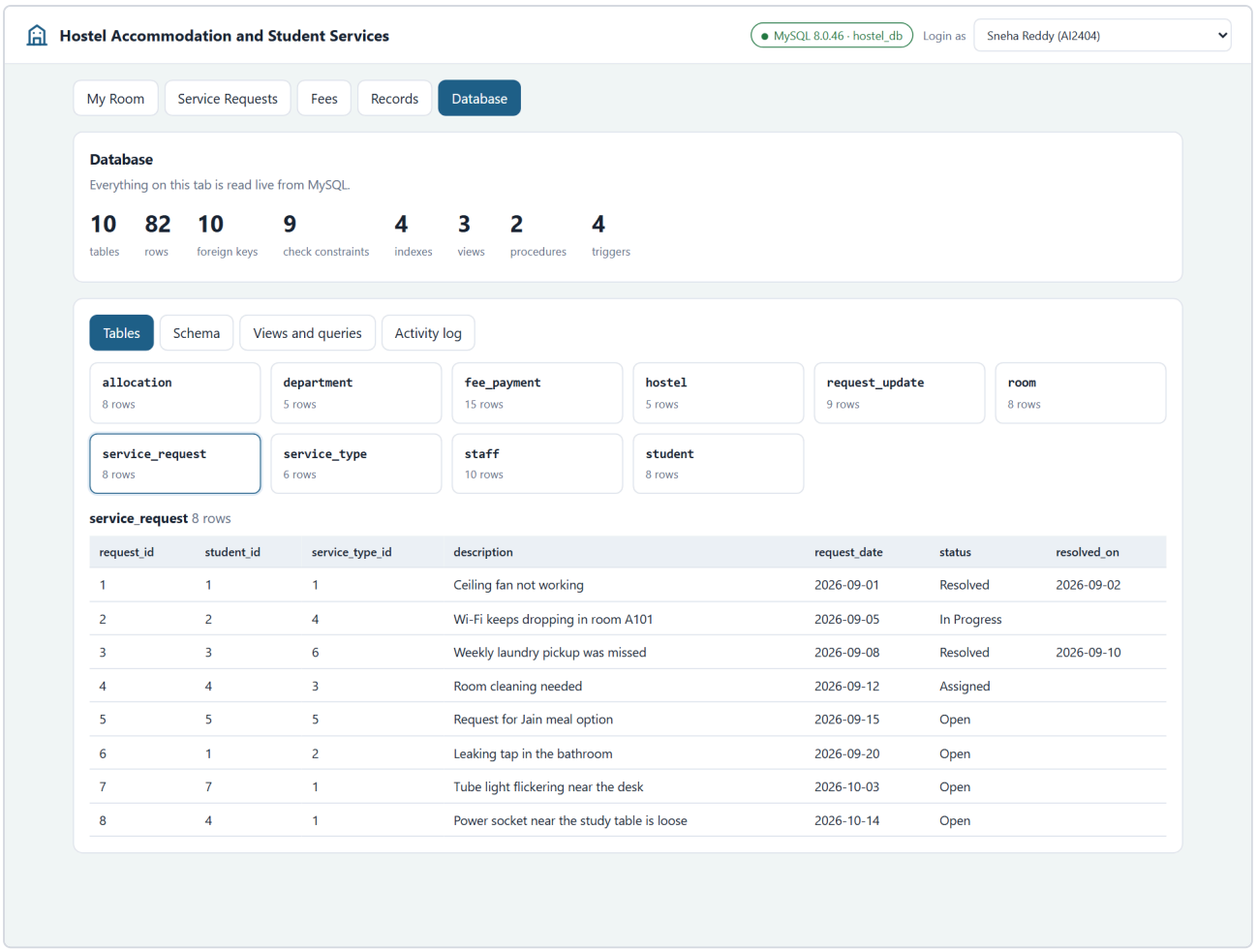
INSERT Figure 5. After: request submitted

The app confirms request #8 with status Open. The panel shows the CALL sent to MySQL.



INSERT **Figure 6. After: the same table read back from MySQL**

service_request now holds 8 rows, so the new record is stored in the database.



request_id	student_id	service_type_id	description	request_date	status	resolved_on
1	1	1	Ceiling fan not working	2026-09-01	Resolved	2026-09-02
2	2	4	Wi-Fi keeps dropping in room A101	2026-09-05	In Progress	
3	3	6	Weekly laundry pickup was missed	2026-09-08	Resolved	2026-09-10
4	4	3	Room cleaning needed	2026-09-12	Assigned	
5	5	5	Request for Jain meal option	2026-09-15	Open	
6	1	2	Leaking tap in the bathroom	2026-09-20	Open	
7	7	1	Tube light flickering near the desk	2026-10-03	Open	
8	4	1	Power socket near the study table is loose	2026-10-14	Open	

MYSQL **MySQL output for Figure 6**

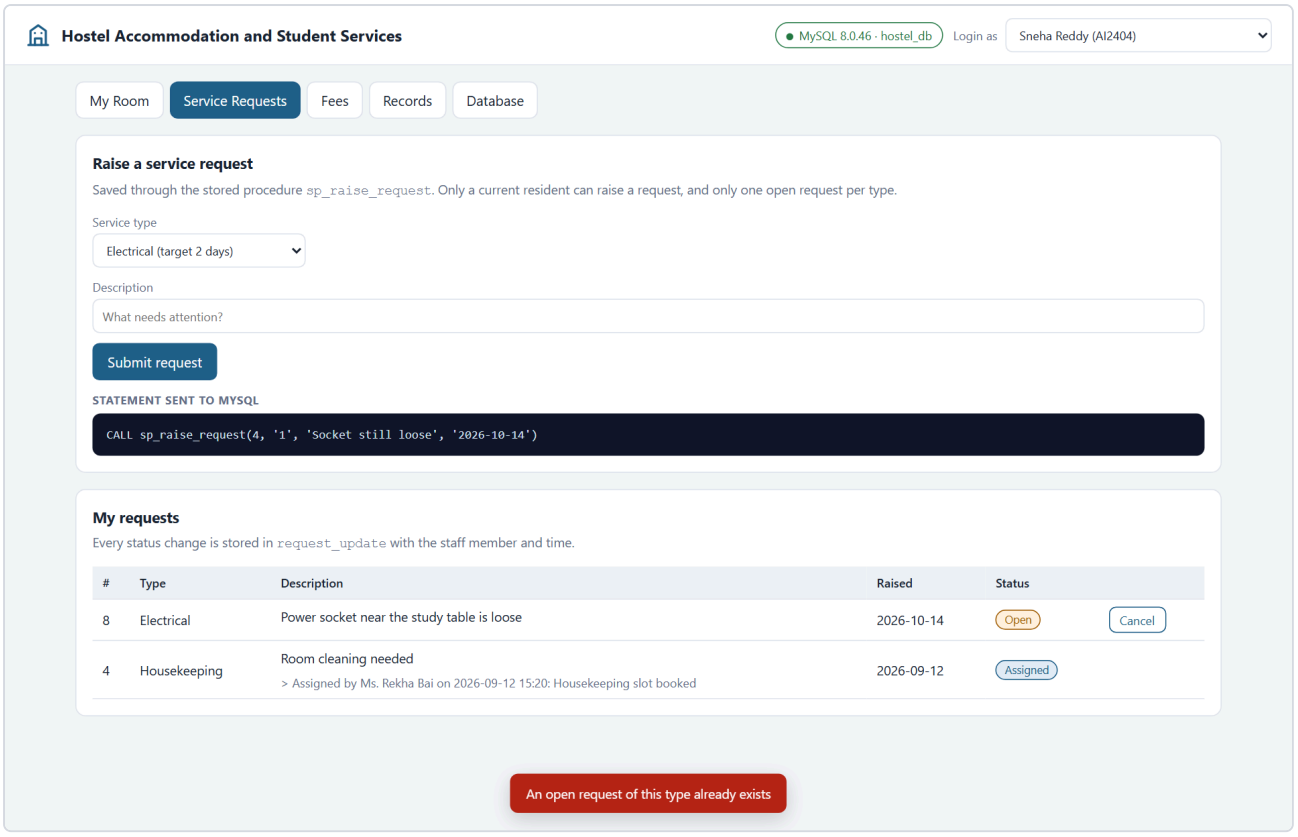
service_request after the insert: request 8 (Sneha Reddy, Electrical, Open).

MySQL command line: hostel_db

```
mysql> SELECT request_id, student_id, service_type_id, description, request_date, status
    -> FROM service_request
    -> ORDER BY request_id DESC LIMIT 3;
+------------+------------+-----------------+-----------------------------------------+--------------+--------+
| request_id | student_id | service_type_id | description                             | request_date | status |
+------------+------------+-----------------+-----------------------------------------+--------------+--------+
|          8 |          4 |               1 | Power socket near the study table is loose | 2026-10-14   | Open   |
|          7 |          7 |               1 | Tube light flickering near the desk        | 2026-10-03   | Open   |
|          6 |          1 |               2 | Leaking tap in the bathroom                | 2026-09-20   | Open   |
+------------+------------+-----------------+-----------------------------------------+--------------+--------+
```

RULE **Figure 7. A second open request of the same type is rejected**

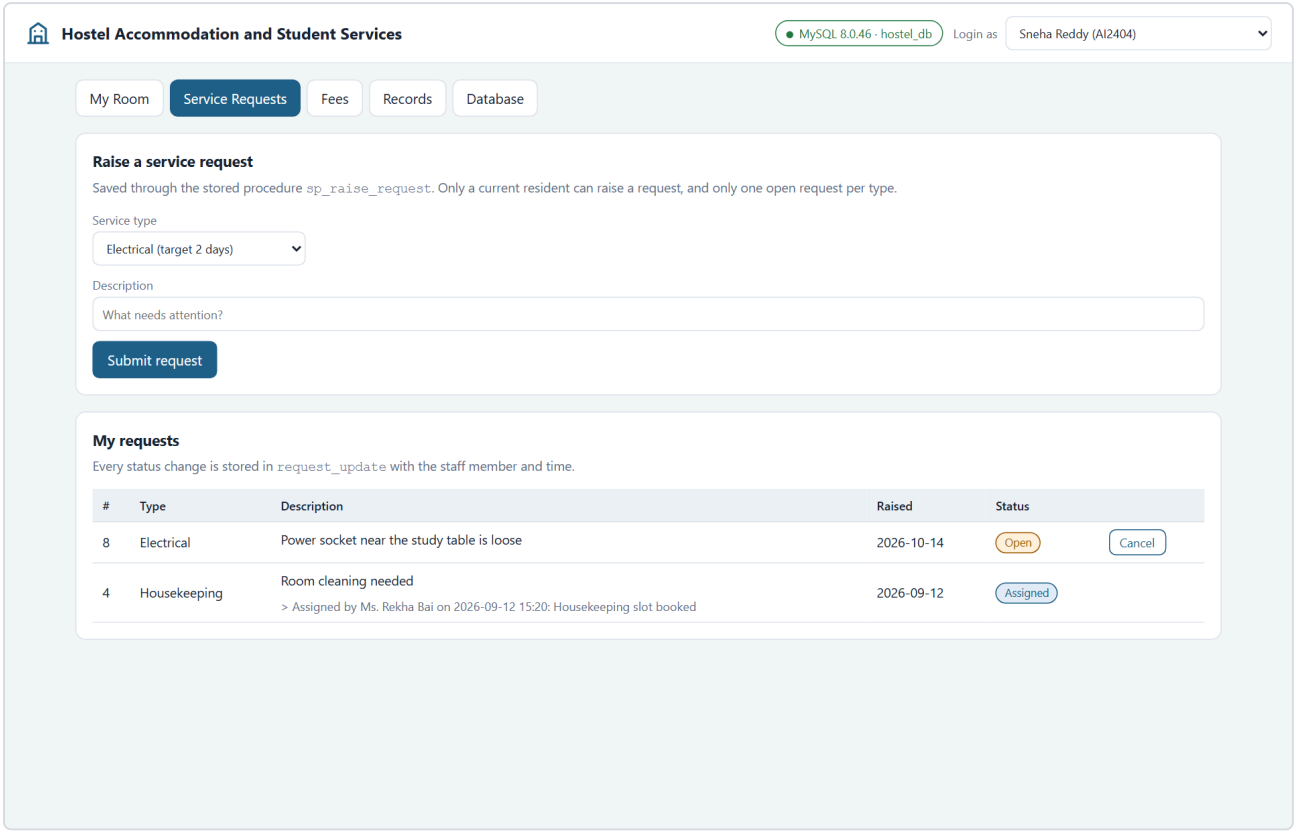
Sneha already has an open Electrical request. The stored procedure sp_raise_request refuses the new one and the message from MySQL is shown.



3. Delete: cancel an open request

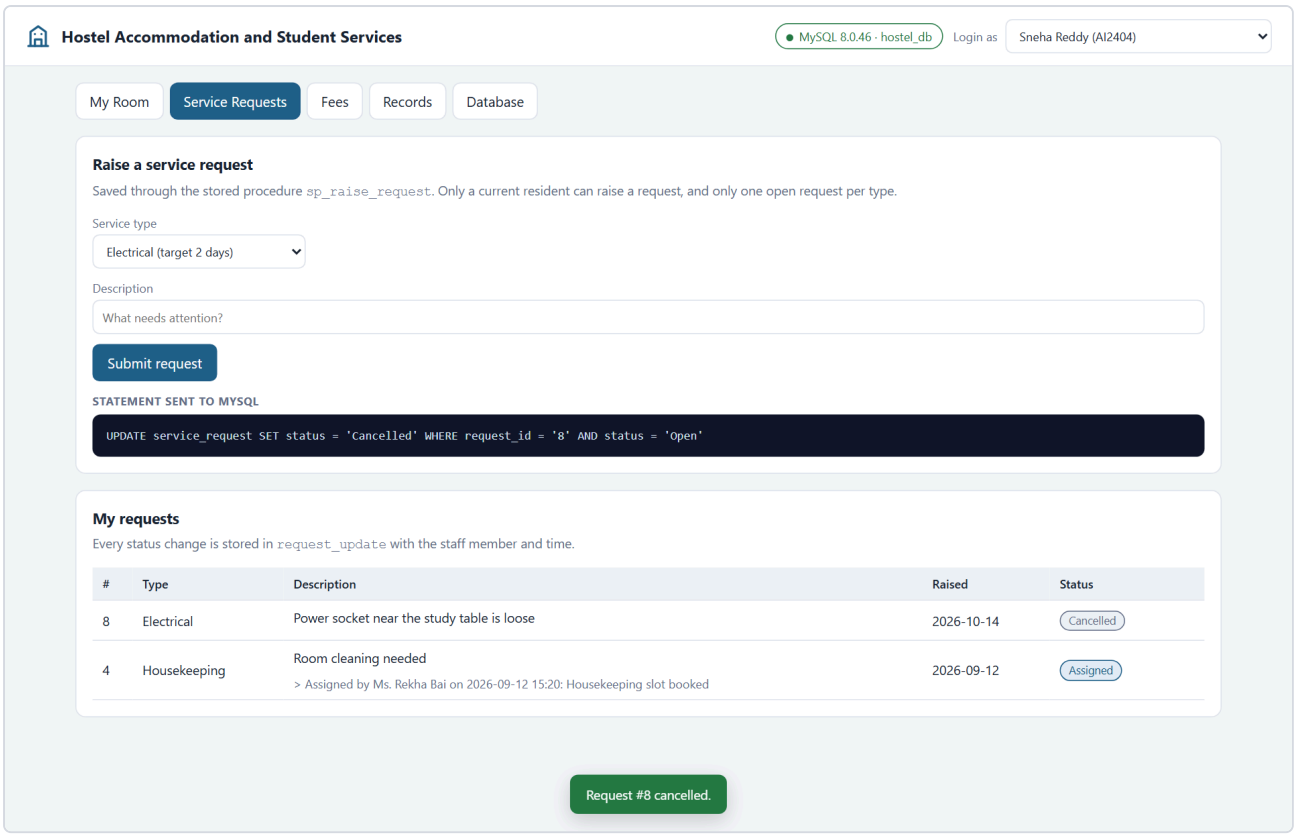
DELETE **Figure 8. Before: My requests**

Request #8 is Open and has a Cancel button.



DELETE Figure 9. After: request cancelled

The status is now Cancelled. The panel shows the UPDATE sent to MySQL. The row is kept so the history stays on record.



MYSQL MySQL output for Figure 9

Request 8 after the cancel: status Cancelled.

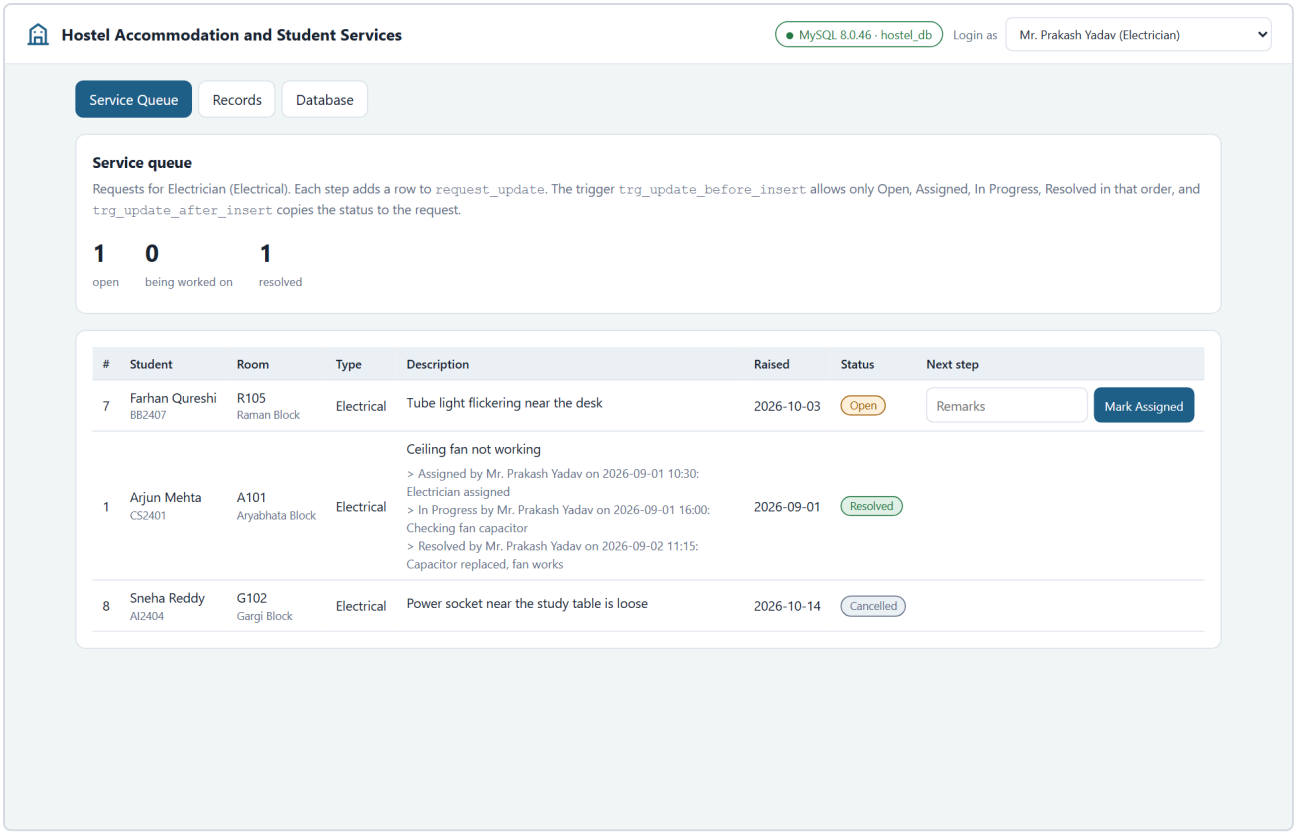
MySQL command line: hostel_db

```
mysql> SELECT request_id, description, status FROM service_request WHERE request_id = 8;
+------------+-----------------------------------------+-----------+
| request_id | description                             | status    |
+------------+-----------------------------------------+-----------+
|          8 | Power socket near the study table is loose | Cancelled |
+------------+-----------------------------------------+-----------+
```

4. Service workflow: Open, Assigned, In Progress, Resolved

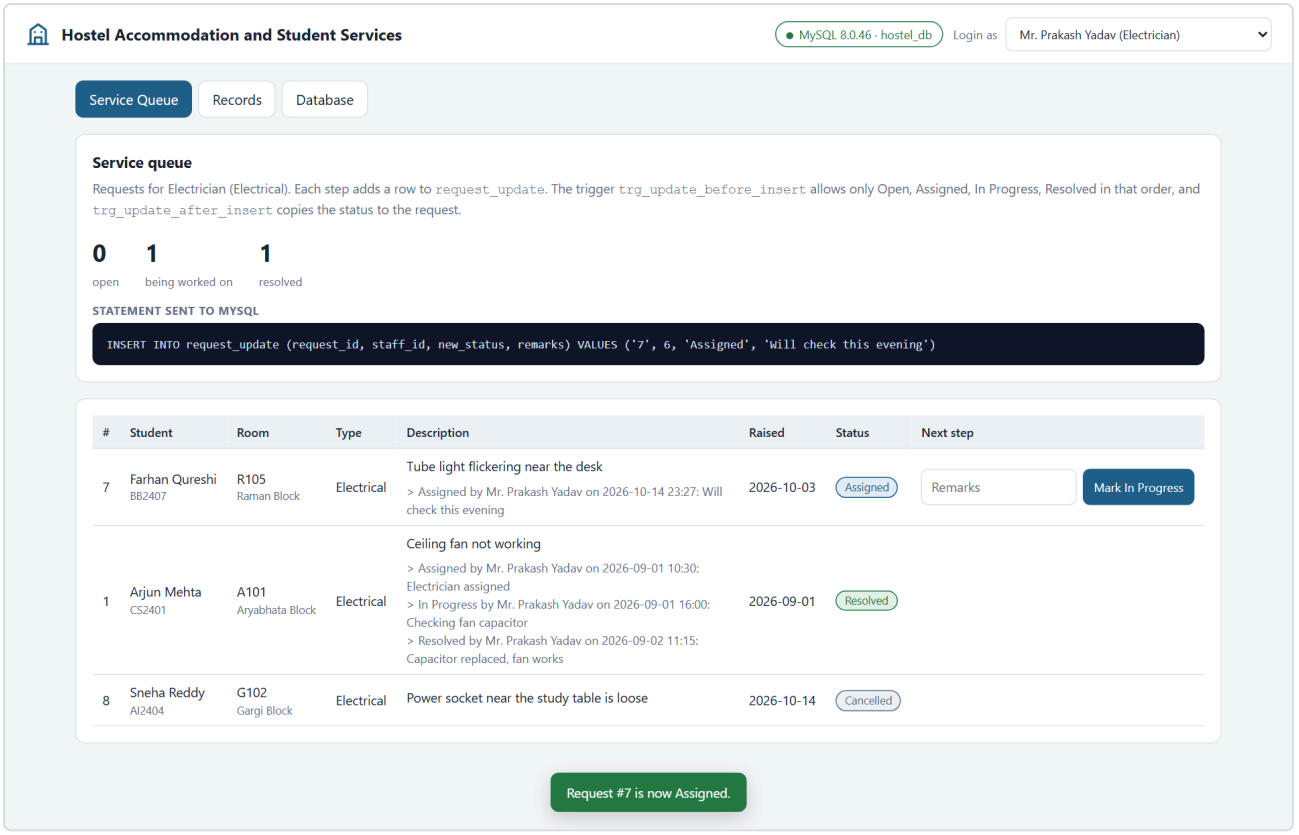
VIEW **Figure 10. Service Queue as the electrician (Mr. Prakash Yadav)**

Only Electrical requests are listed. Request #7 from Farhan Qureshi is Open.



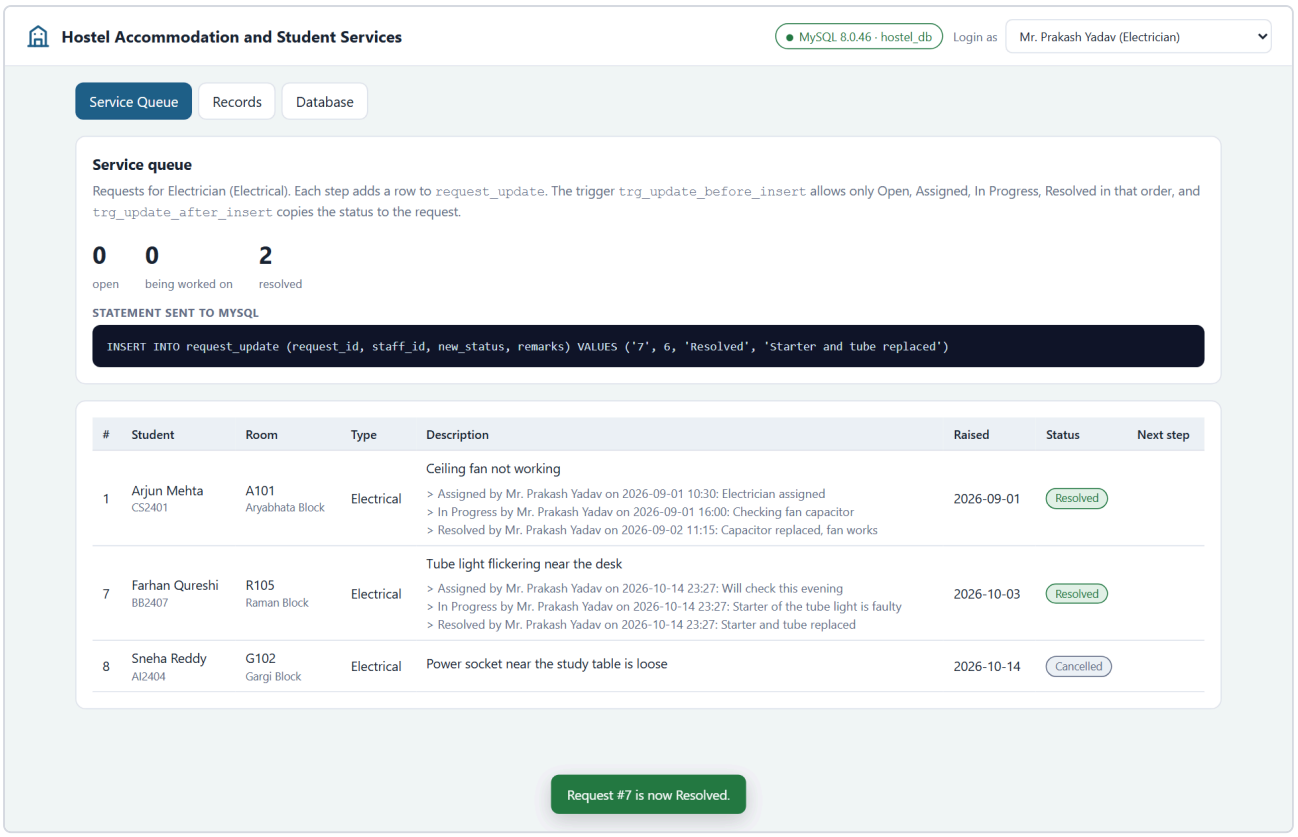
INSERT **Figure 11. After the first step: Assigned**

A row was inserted into request_update and the trigger trg_update_after_insert moved request #7 to Assigned.



INSERT Figure 12. After the last step: Resolved

Request #7 is Resolved. The trigger also wrote the resolved date.



MYSQL MySQL output for Figure 12

request_update has three rows for request 7, and the request itself is Resolved with resolved_on filled in by the trigger.

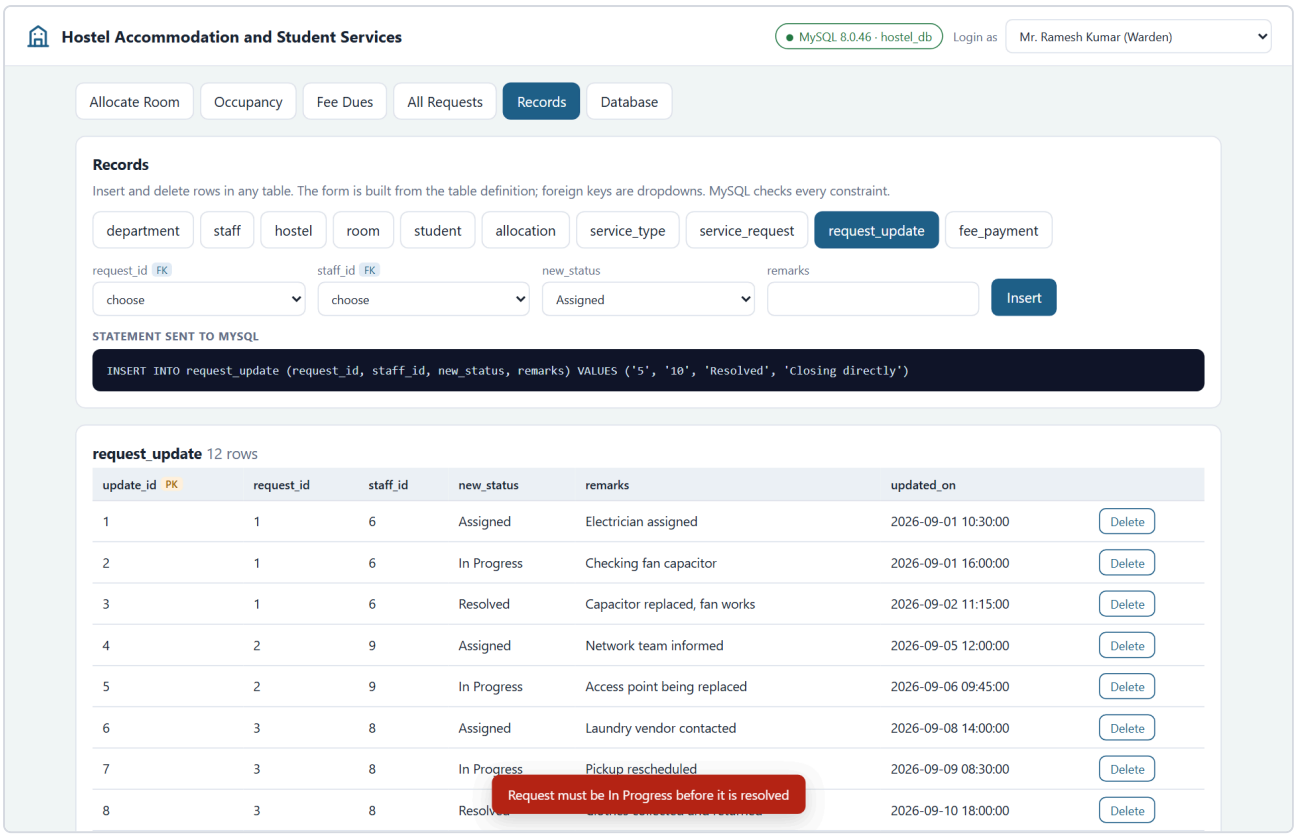
MySQL command line: hostel_db

```
mysql> SELECT ru.update_id, ru.request_id, ru.new_status, ru.remarks, ru.updated_on
    -> FROM request_update ru WHERE ru.request_id = 7;
+-----------+------------+-------------+-----------------------------------+---------------------+
| update_id | request_id | new_status  | remarks                           | updated_on          |
+-----------+------------+-------------+-----------------------------------+---------------------+
|        10 |          7 | Assigned    | Will check this evening           | 2026-10-14 23:27:57 |
|        11 |          7 | In Progress | Starter of the tube light is faulty | 2026-10-14 23:27:58 |
|        12 |          7 | Resolved    | Starter and tube replaced         | 2026-10-14 23:27:59 |
+-----------+------------+-------------+-----------------------------------+---------------------+

mysql> SELECT request_id, status, resolved_on FROM service_request WHERE request_id = 7;
+------------+----------+-------------+
| request_id | status   | resolved_on |
+------------+----------+-------------+
|          7 | Resolved | 2026-10-14  |
+------------+----------+-------------+
```

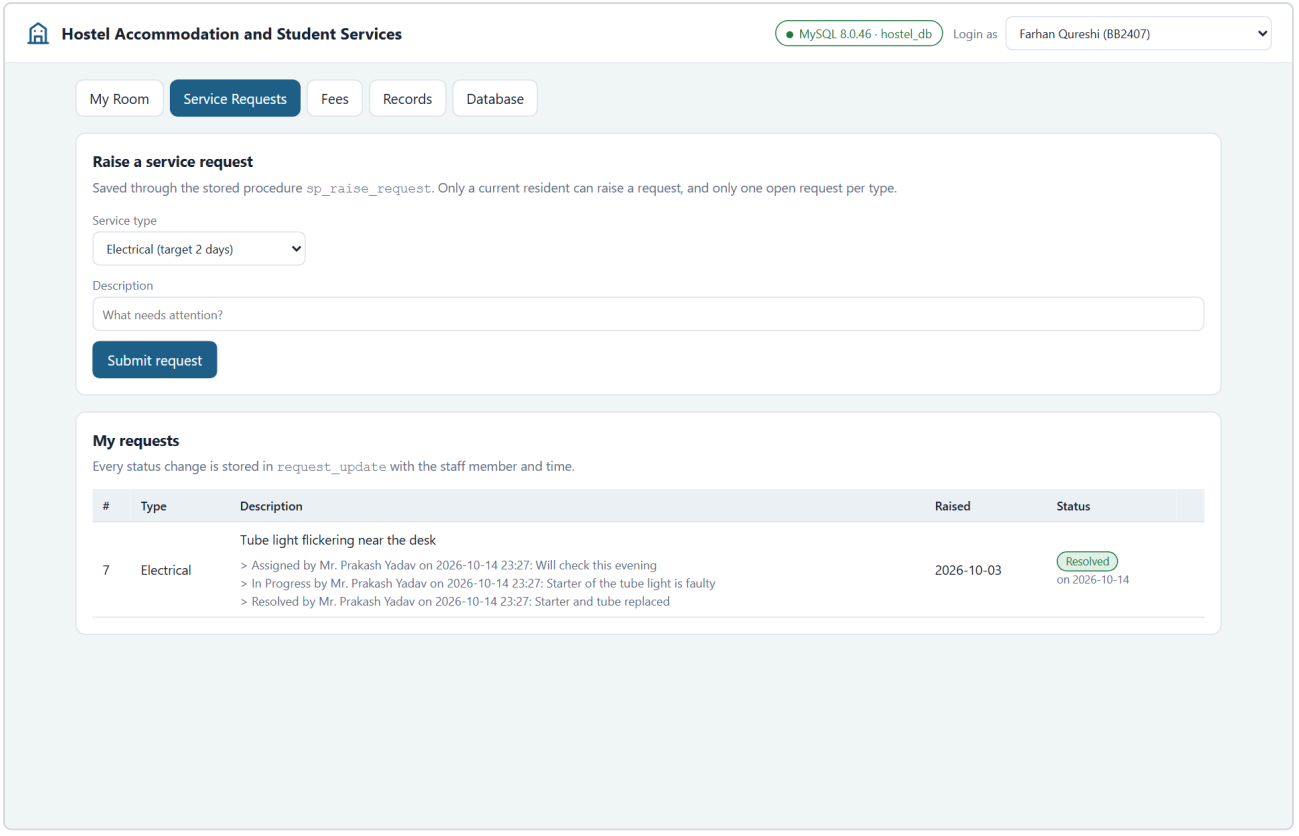
RULE **Figure 13. Skipping a step is refused**

A direct insert that marks the Open request #5 as Resolved is stopped by the trigger trg_update_before_insert.



VIEW **Figure 14. Farhan Qureshi's requests after the work is done**

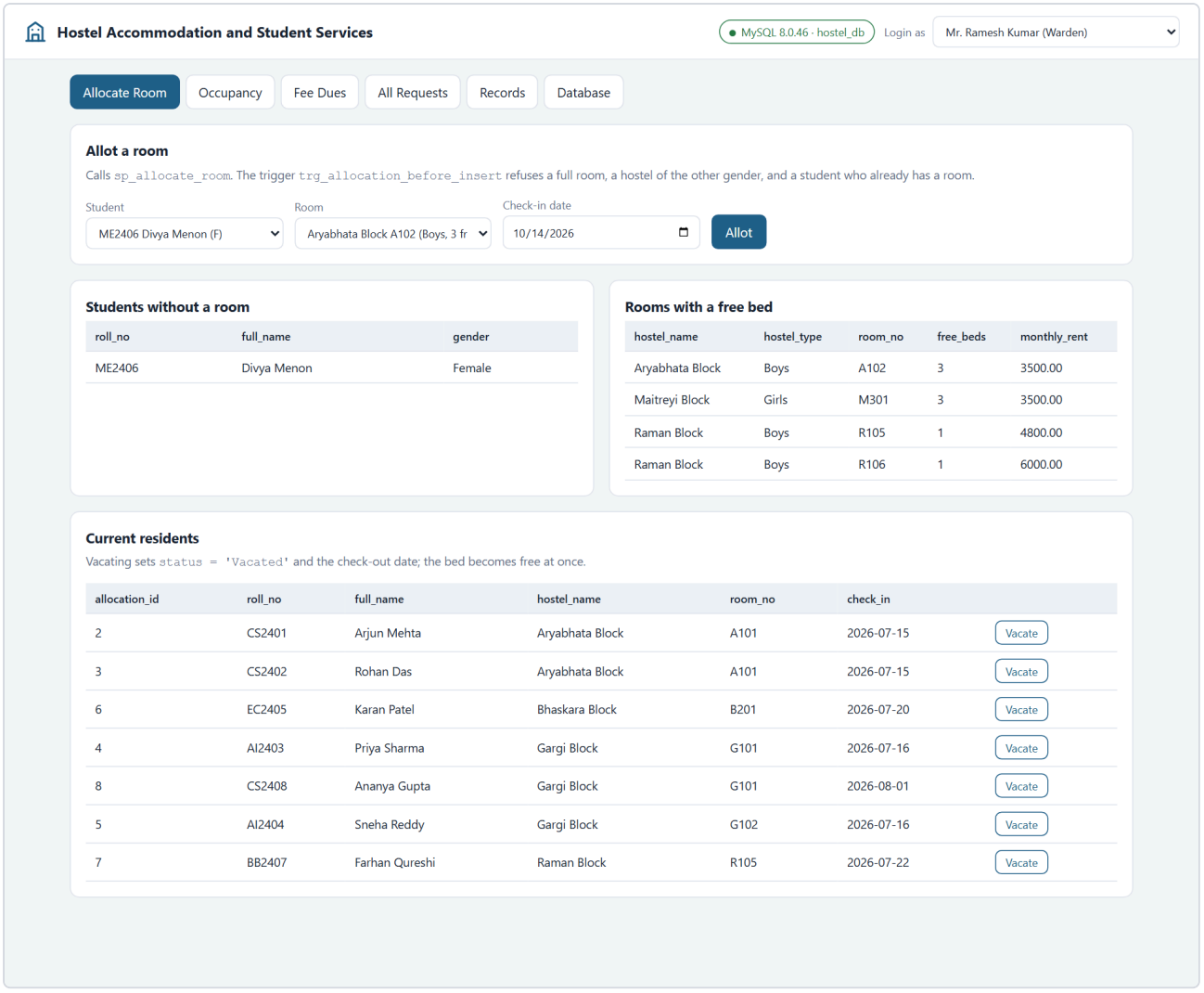
The student sees request #7 as Resolved with the full trail.



5. Room allotment by the warden

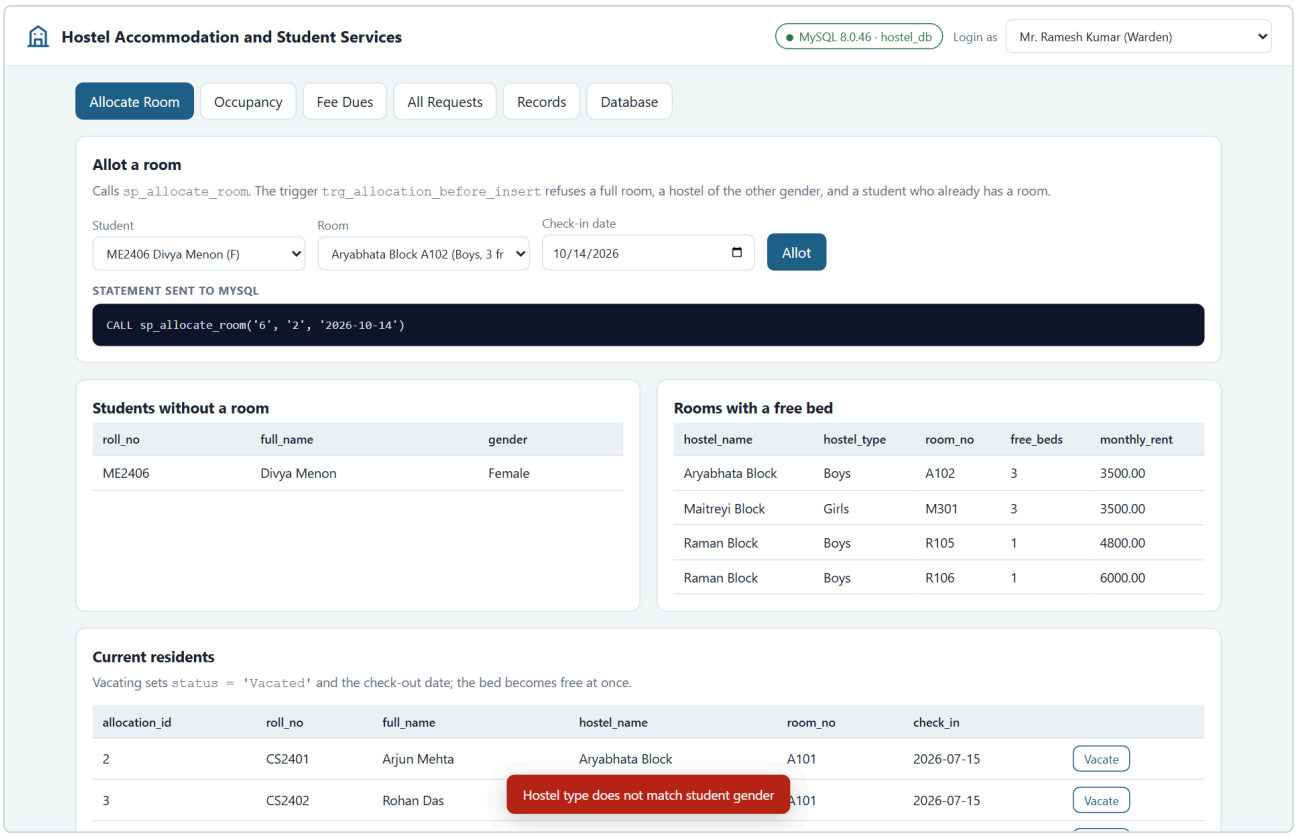
VIEW Figure 15. Allocate Room as the warden (Mr. Ramesh Kumar)

Divya Menon (Female) has no room. Rooms with a free bed are listed on the right.



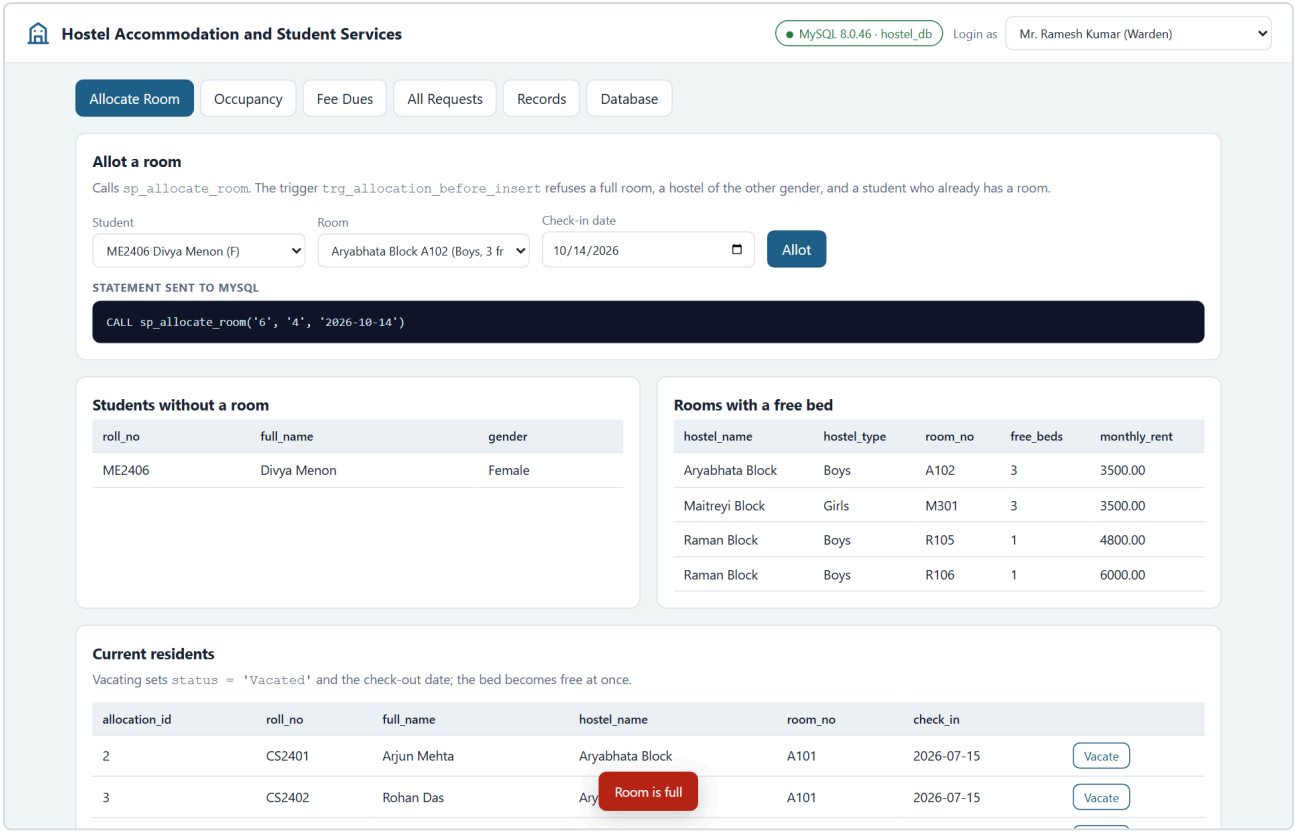
RULE Figure 16. A boys' hostel room is refused for a female student

Divya cannot be put in A102 of Aryabhata Block. The trigger trg_allocation_before_insert raises the error.



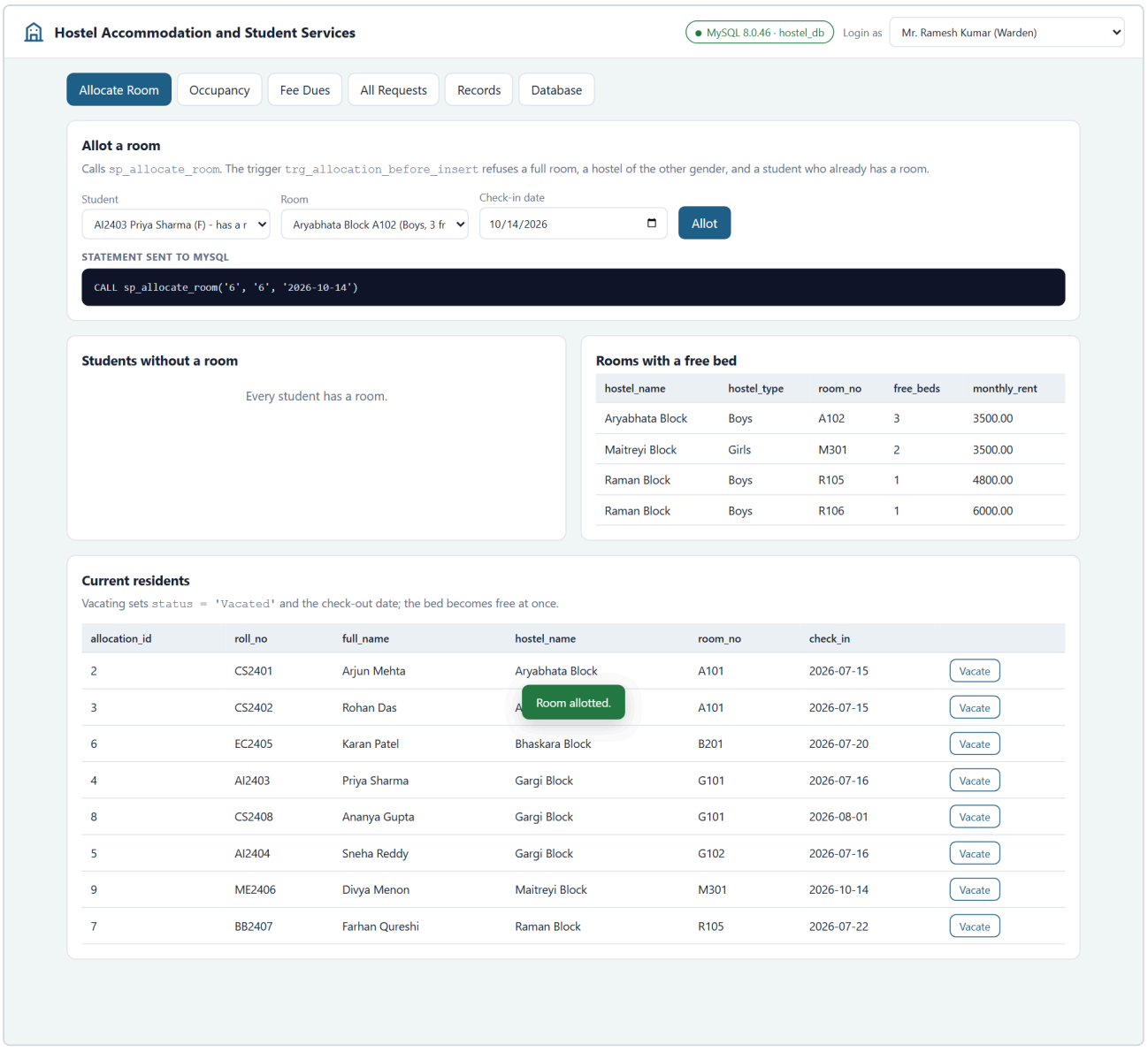
RULE Figure 17. A full room is refused

G101 of Gargi Block already has two residents in a double room, so MySQL answers "Room is full".



INSERT Figure 18. After: Divya Menon allotted to M301

The procedure sp_allocate_room inserted the allocation. Divya no longer appears under students without a room.



MYSQL MySQL output for Figure 18

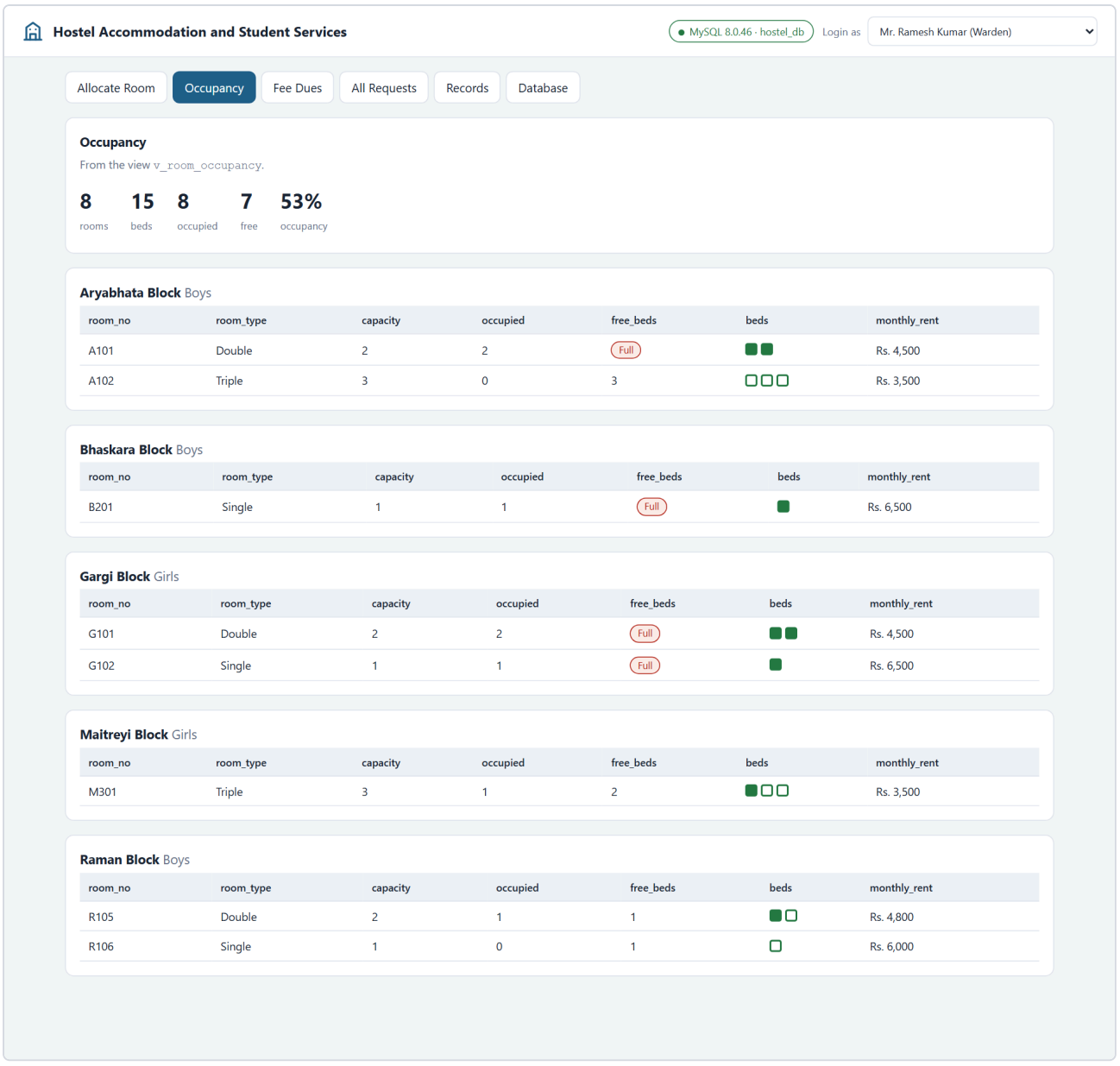
Divya has her old Vacated stay and the new Active one.

MySQL command line: hostel_db

```
mysql> SELECT a.allocation_id, s.full_name, r.room_no, a.check_in, a.status
    -> FROM allocation a JOIN student s ON s.student_id = a.student_id
    -> JOIN room r ON r.room_id = a.room_id
    -> WHERE s.student_id = 6;
+---------------+-------------+---------+------------+---------+
| allocation_id | full_name   | room_no | check_in   | status  |
+---------------+-------------+---------+------------+---------+
|             9 | Divya Menon | M301    | 2026-10-14 | Active  |
|             1 | Divya Menon | M301    | 2025-07-10 | Vacated |
+---------------+-------------+---------+------------+---------+
```

VIEW Figure 19. Occupancy after the allotment

Built from the view v_room_occupancy. M301 now shows 1 of 3 beds taken. Full rooms are marked in red.



MYSQL MySQL output for Figure 19

The view gives the same numbers as the page.

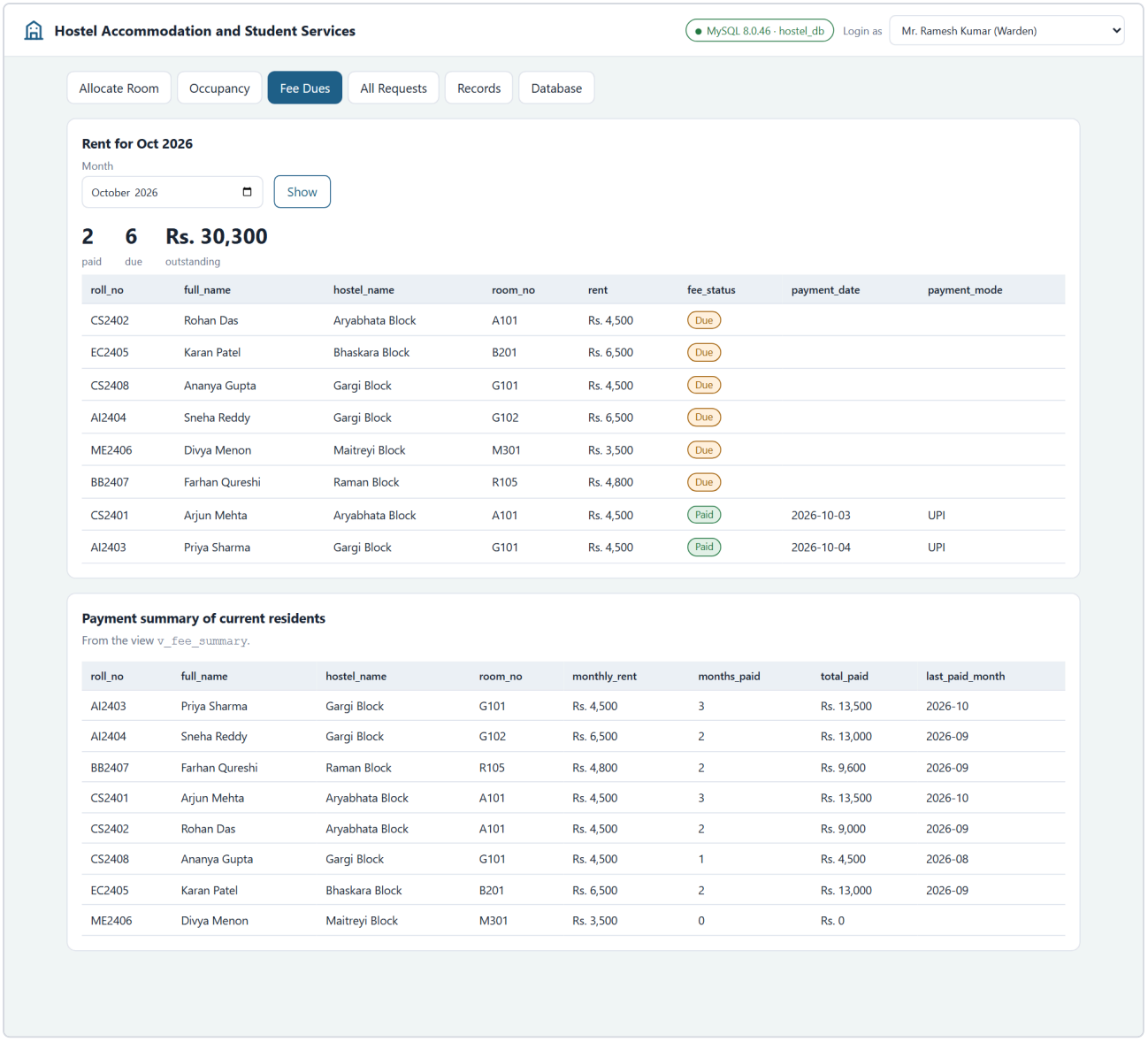
MySQL command line: hostel_db

```
mysql> SELECT hostel_name, room_no, capacity, occupied, free_beds
    -> FROM v_room_occupancy ORDER BY hostel_name, room_no;
+----------------+---------+----------+----------+-----------+
| hostel_name    | room_no | capacity | occupied | free_beds |
+----------------+---------+----------+----------+-----------+
| Aryabhata Block | A101    |        2 |        2 |         0 |
| Aryabhata Block | A102    |        3 |        0 |         3 |
| Bhaskara Block  | B201    |        1 |        1 |         0 |
| Gargi Block     | G101    |        2 |        2 |         0 |
| Gargi Block     | G102    |        1 |        1 |         0 |
| Maitreyi Block  | M301    |        3 |        1 |         2 |
| Raman Block     | R105    |        2 |        1 |         1 |
| Raman Block     | R106    |        1 |        0 |         1 |
+----------------+---------+----------+----------+-----------+
```

6. Rent payment

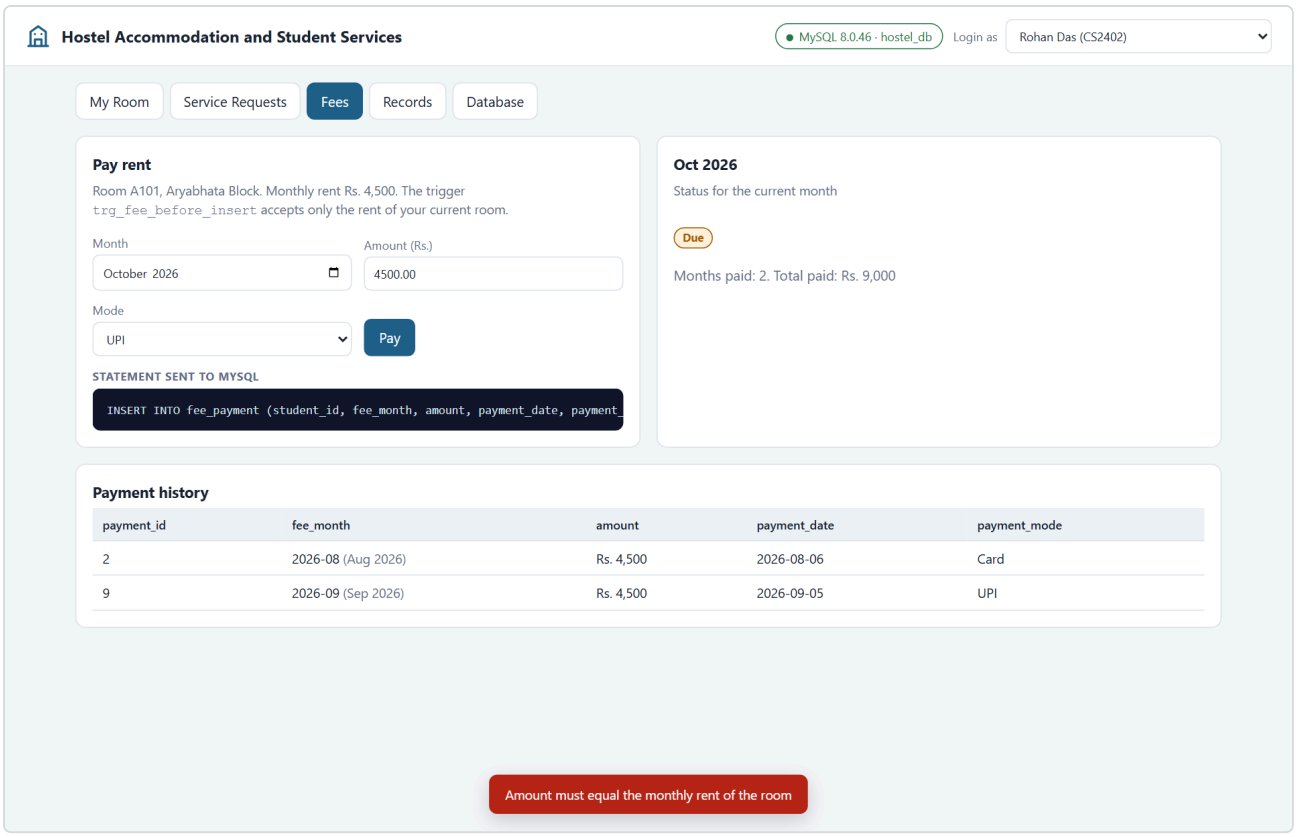
VIEW Figure 20. Fee Dues for October 2026 (before)

Five residents have not paid October rent, including Rohan Das.



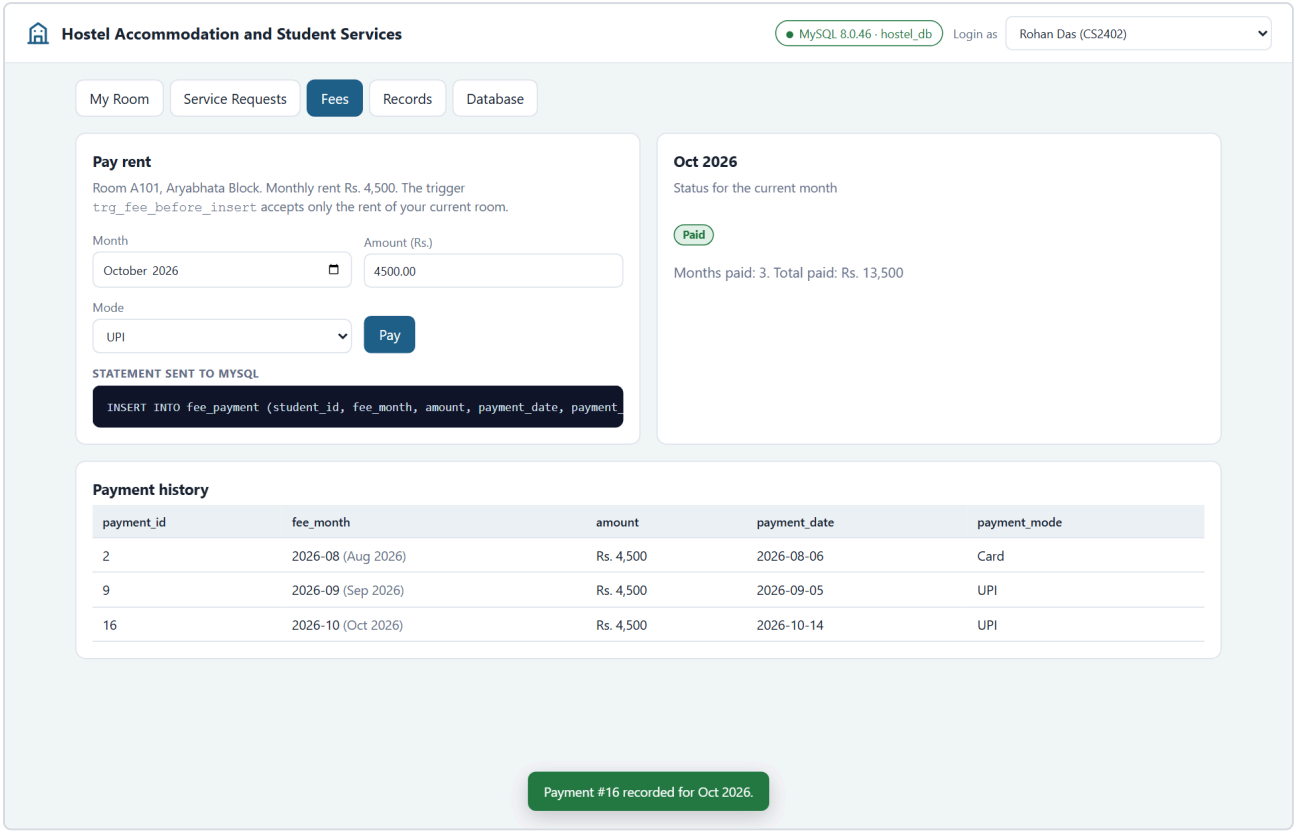
RULE **Figure 21. A wrong amount is refused**

Rohan tries to pay Rs. 4,000. The trigger trg_fee_before_insert accepts only the rent of his room, Rs. 4,500.



INSERT **Figure 22. After: October rent paid**

Payment saved. The month now shows Paid and the payment history has a new row.



MYSQL MySQL output for Figure 22

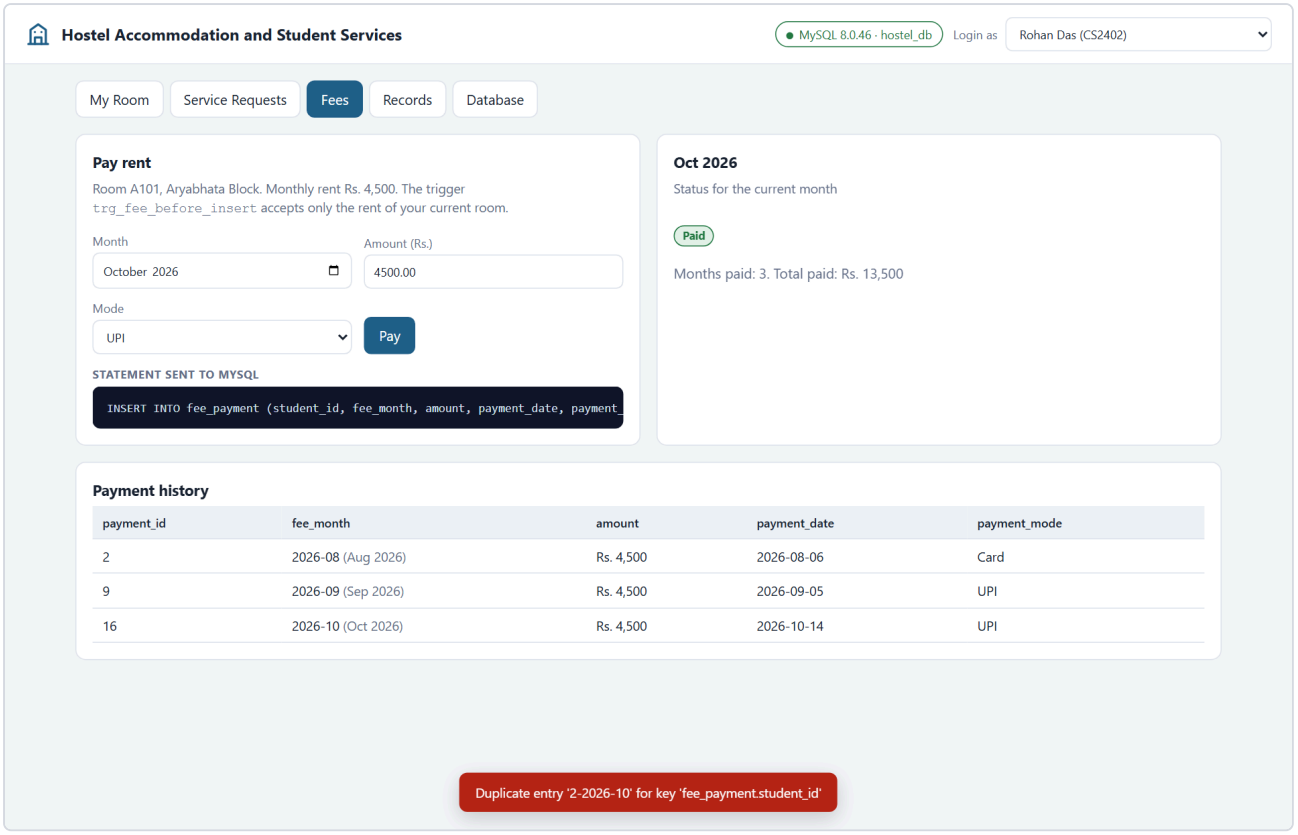
fee_payment for Rohan Das: August, September and the new October row.

MySQL command line: hostel_db

```
mysql> SELECT payment_id, student_id, fee_month, amount, payment_date, payment_mode
    -> FROM fee_payment WHERE student_id = 2;
+------------+------------+-----------+---------+--------------+--------------+
| payment_id | student_id | fee_month | amount  | payment_date | payment_mode |
+------------+------------+-----------+---------+--------------+--------------+
|          2 |          2 | 2026-08   | 4500.00 | 2026-08-06   | Card         |
|          9 |          2 | 2026-09   | 4500.00 | 2026-09-05   | UPI          |
|         16 |          2 | 2026-10   | 4500.00 | 2026-10-14   | UPI          |
+------------+------------+-----------+---------+--------------+--------------+
```

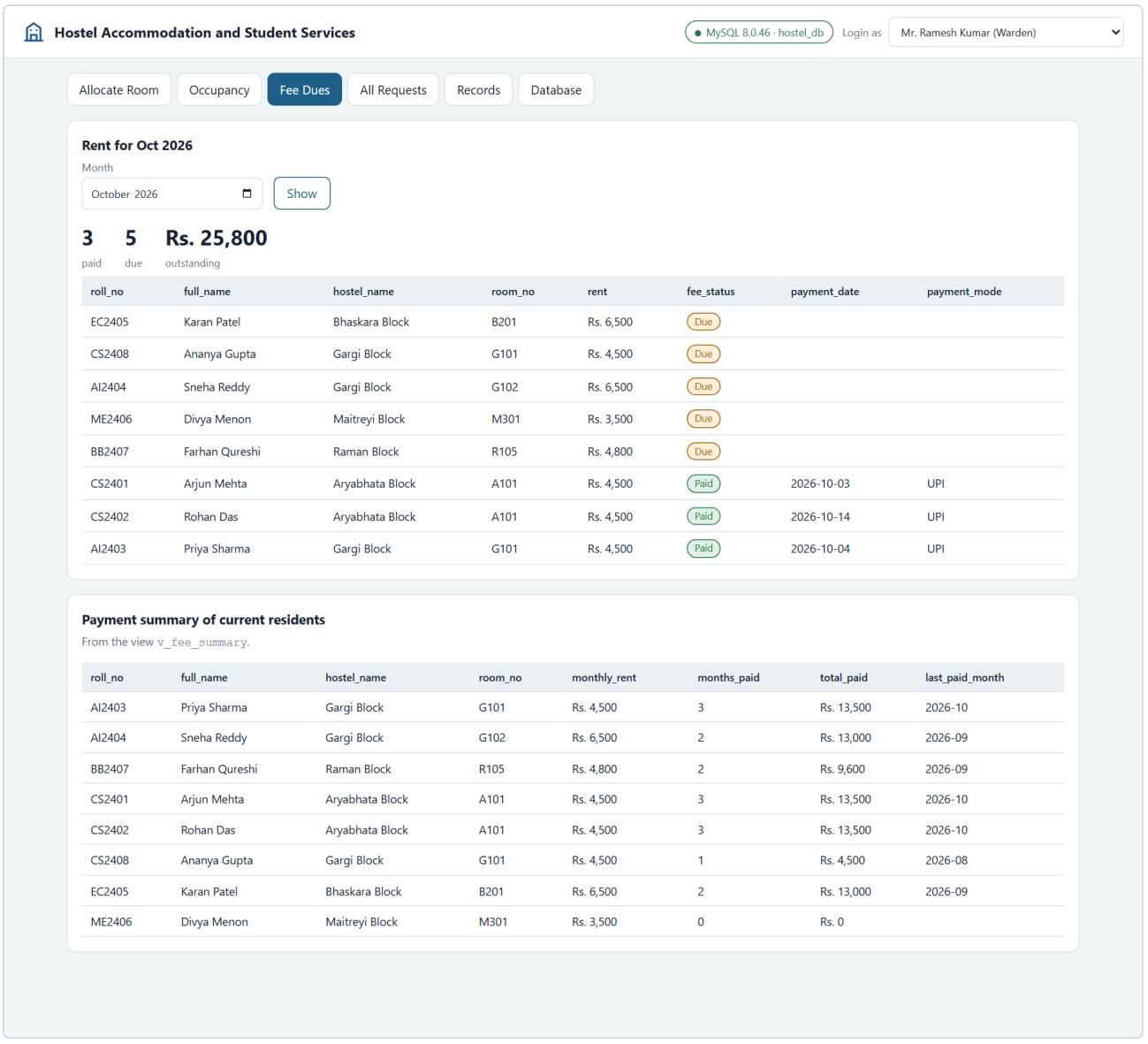
RULE Figure 23. Paying the same month twice is refused

UNIQUE (student_id, fee_month) blocks a second October payment.



VIEW Figure 24. Fee Dues for October 2026 (after)

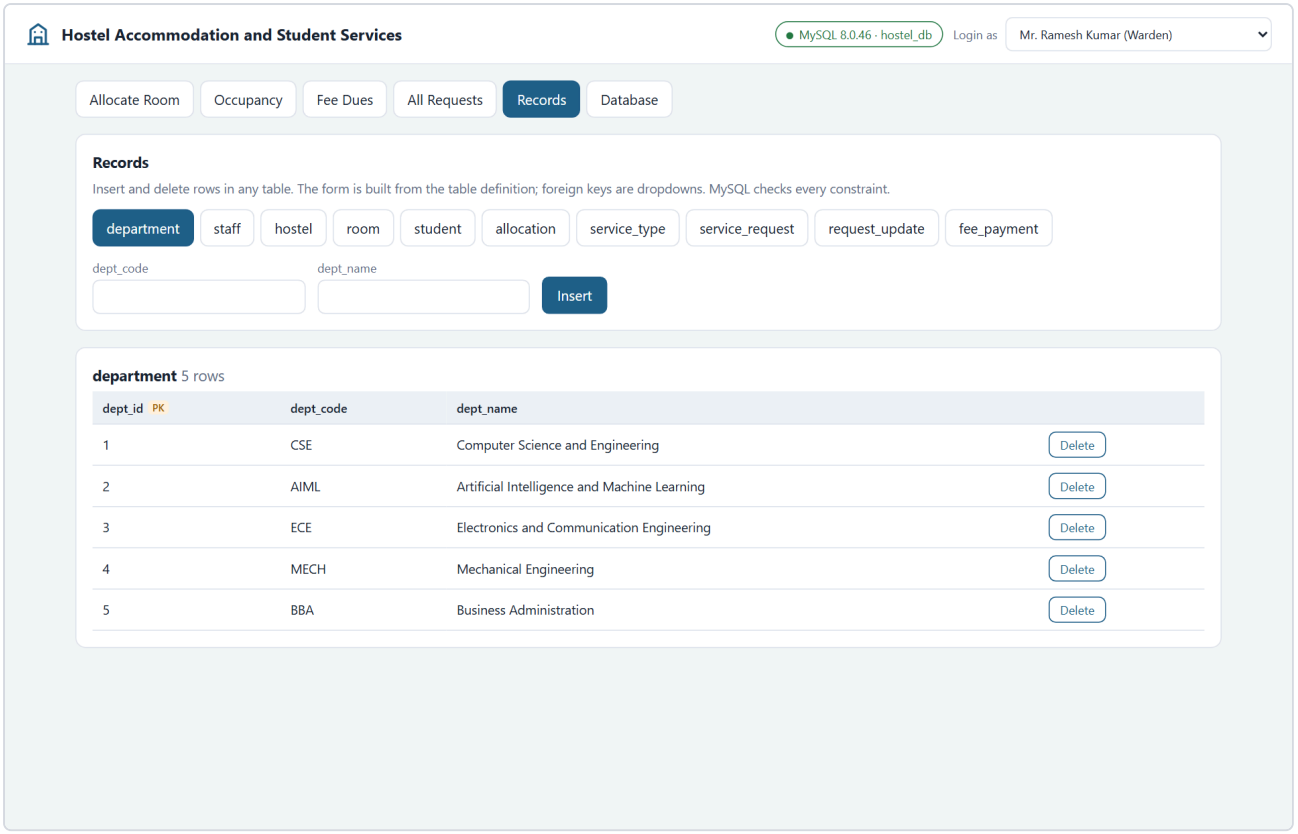
Rohan Das is now Paid. Divya Menon, allotted earlier in this walkthrough, is listed as Due.



7. Insert and delete a record: department

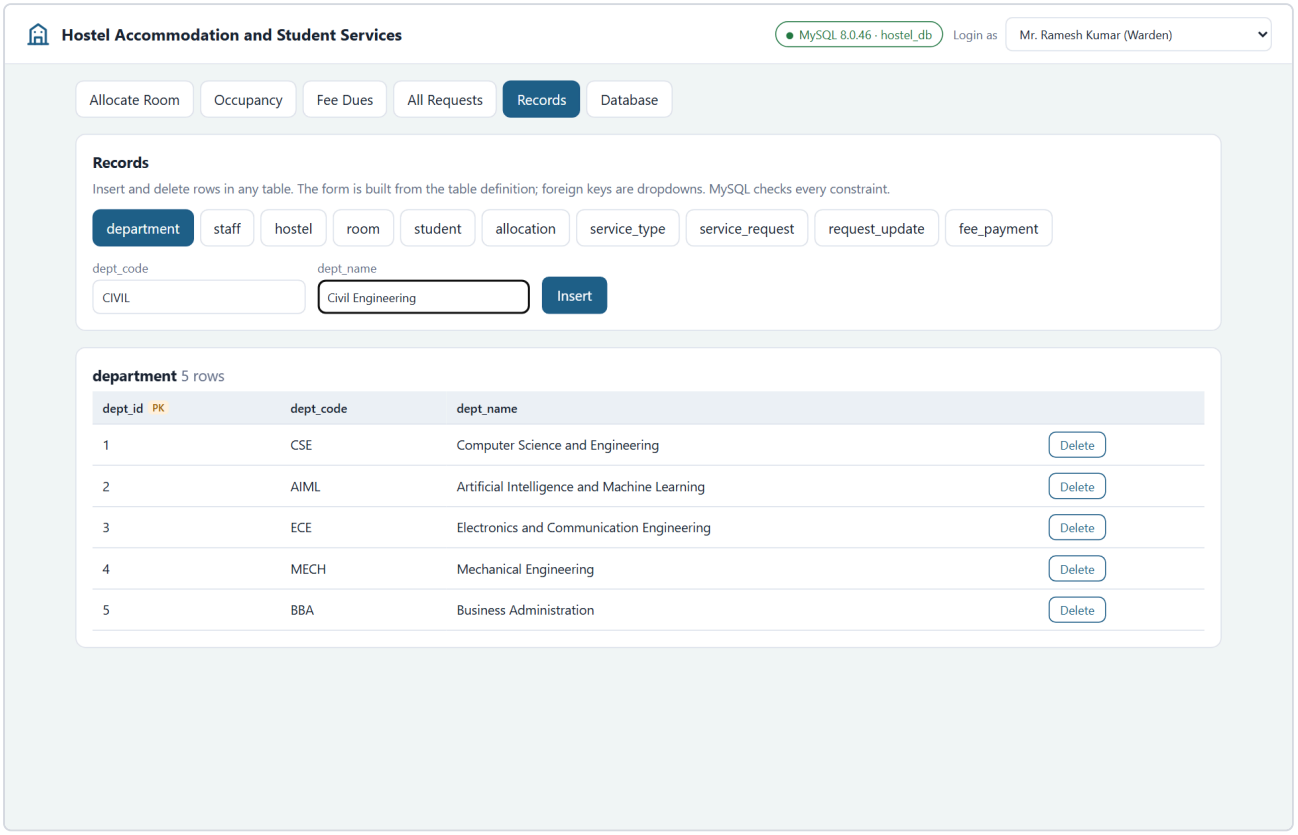
INSERT Figure 25. Before: Records tab, department table

The table has 5 rows. The form is generated from the table definition.



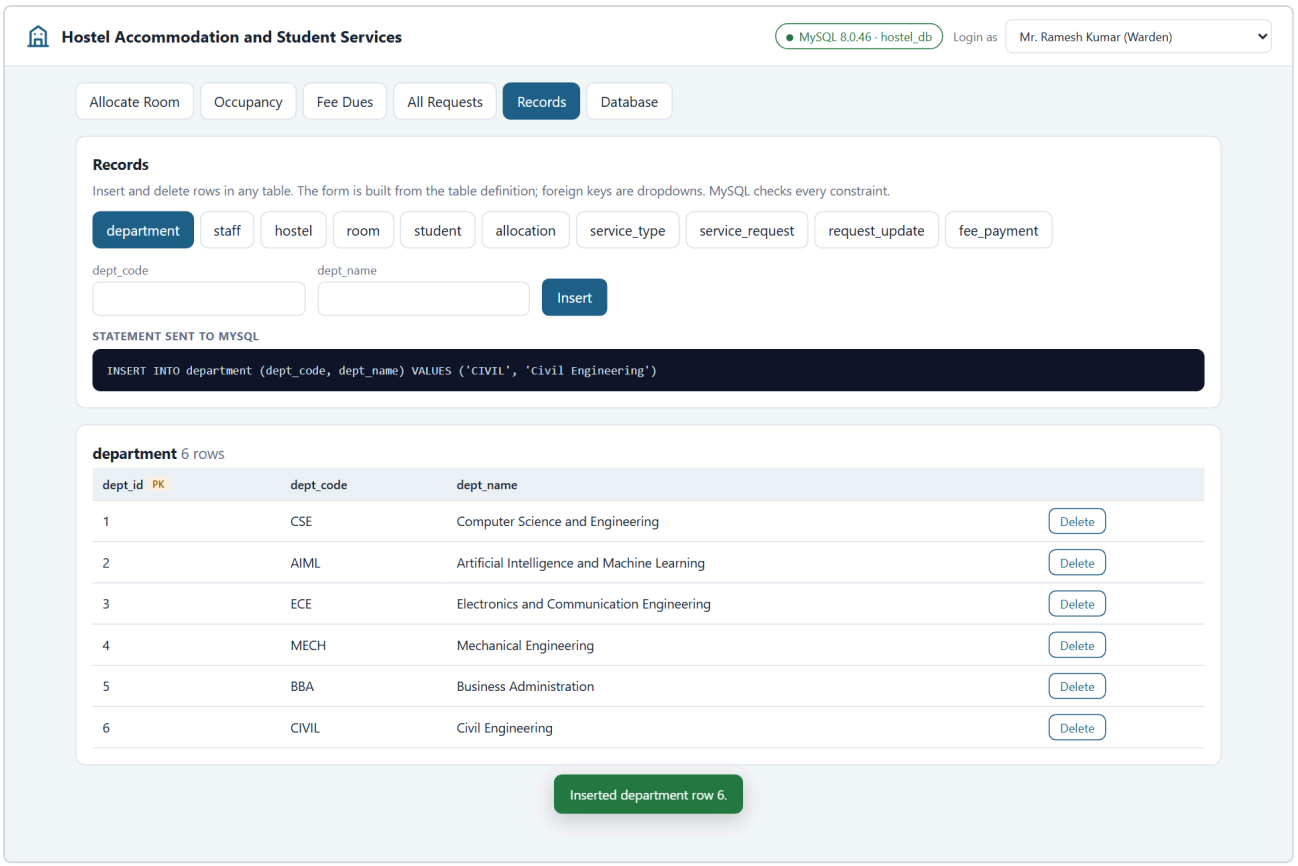
INSERT Figure 26. Insert form filled in

New department "Civil Engineering" with code CIVIL.



INSERT Figure 27. After: record inserted

The table has 6 rows. The panel shows the INSERT statement sent to MySQL.



MYSQL MySQL output for Figure 27

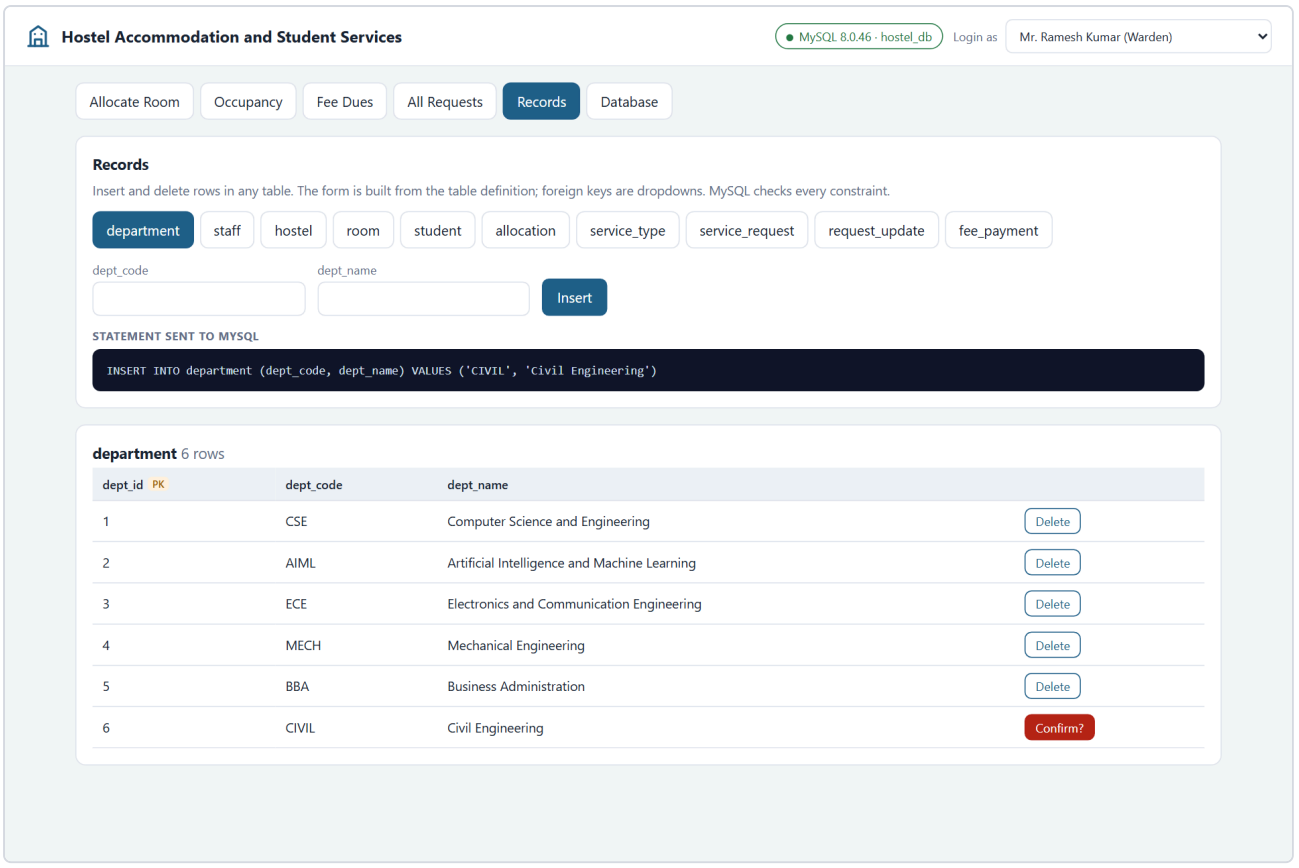
department after the insert: 6 rows, including Civil Engineering with dept_id 6.

MySQL command line: hostel_db

```
mysql> SELECT * FROM department;
+---------+-----------+---------------------------------------------+
| dept_id | dept_code | dept_name                                   |
+---------+-----------+---------------------------------------------+
|       1 | CSE       | Computer Science and Engineering            |
|       2 | AIML      | Artificial Intelligence and Machine Learning |
|       3 | ECE       | Electronics and Communication Engineering    |
|       4 | MECH      | Mechanical Engineering                      |
|       5 | BBA       | Business Administration                     |
|       6 | CIVIL     | Civil Engineering                           |
+---------+-----------+---------------------------------------------+
```

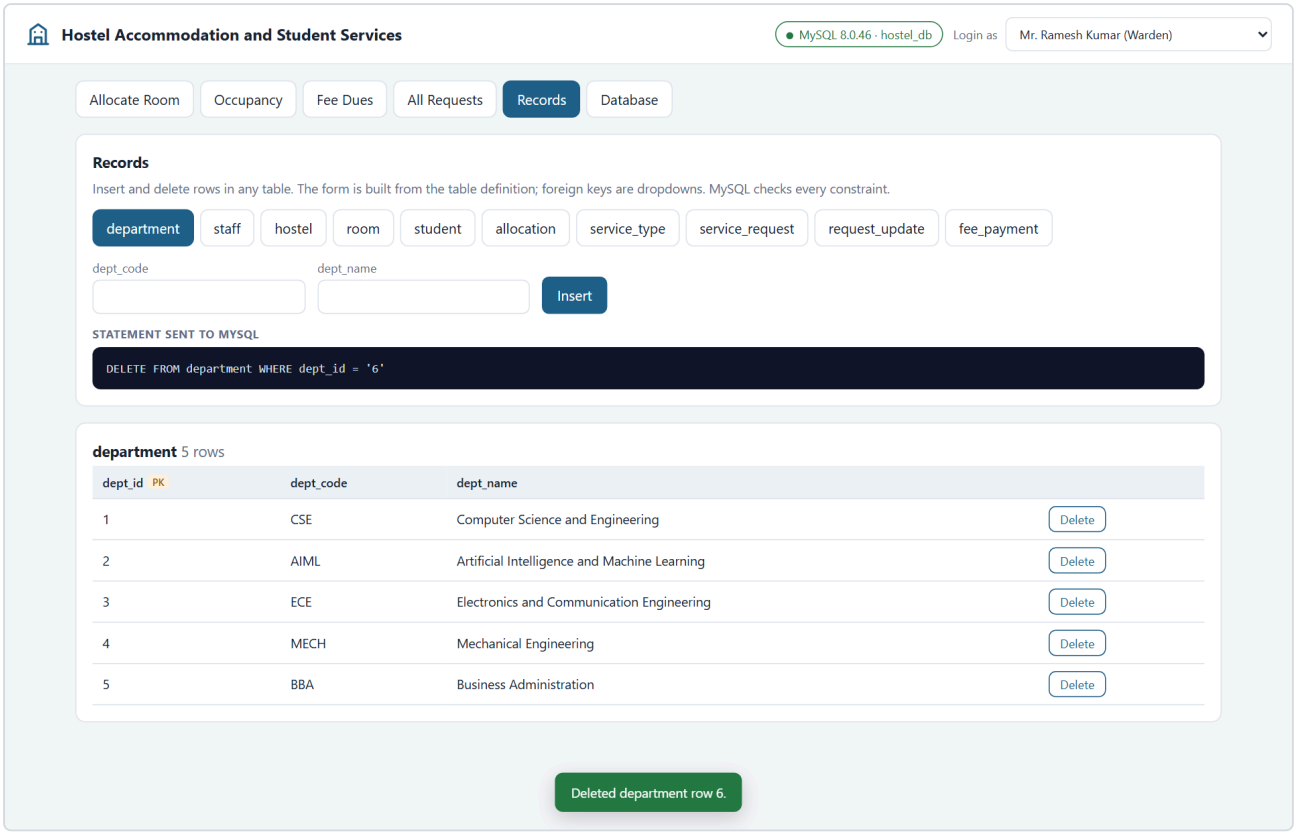
DELETE **Figure 28. Delete asks for confirmation**

The first click turns the button into "Confirm?". A second click runs the delete.



DELETE **Figure 29. After: record deleted**

The table is back to 5 rows. The panel shows the DELETE statement sent to MySQL.



MYSQL MySQL output for Figure 29

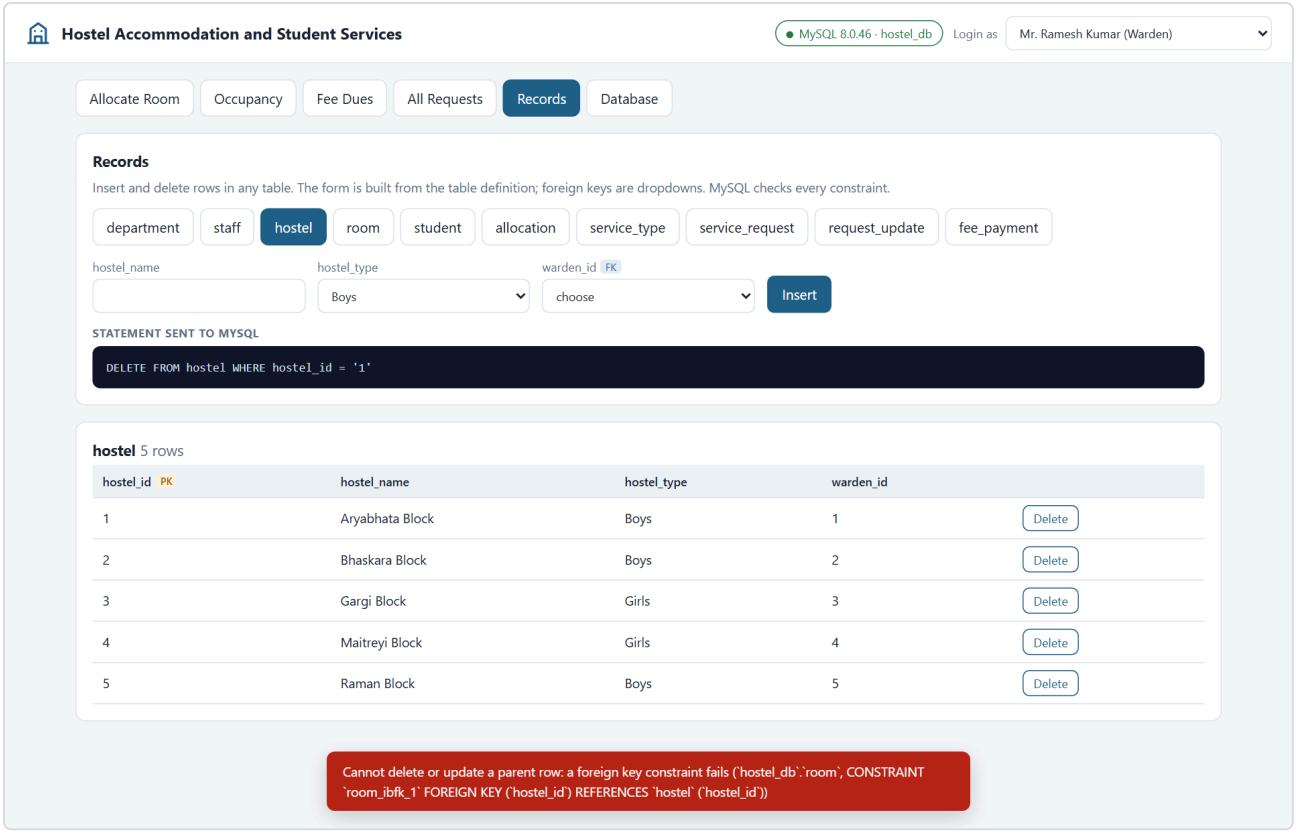
department after the delete: back to 5 rows.

MySQL command line: hostel_db

```
mysql> SELECT * FROM department;
+---------+-----------+-------------------------------------------+
| dept_id | dept_code | dept_name                                 |
+---------+-----------+-------------------------------------------+
|       1 | CSE       | Computer Science and Engineering          |
|       2 | AIML      | Artificial Intelligence and Machine Learning |
|       3 | ECE       | Electronics and Communication Engineering |
|       4 | MECH      | Mechanical Engineering                    |
|       5 | BBA       | Business Administration                   |
+---------+-----------+-------------------------------------------+
```

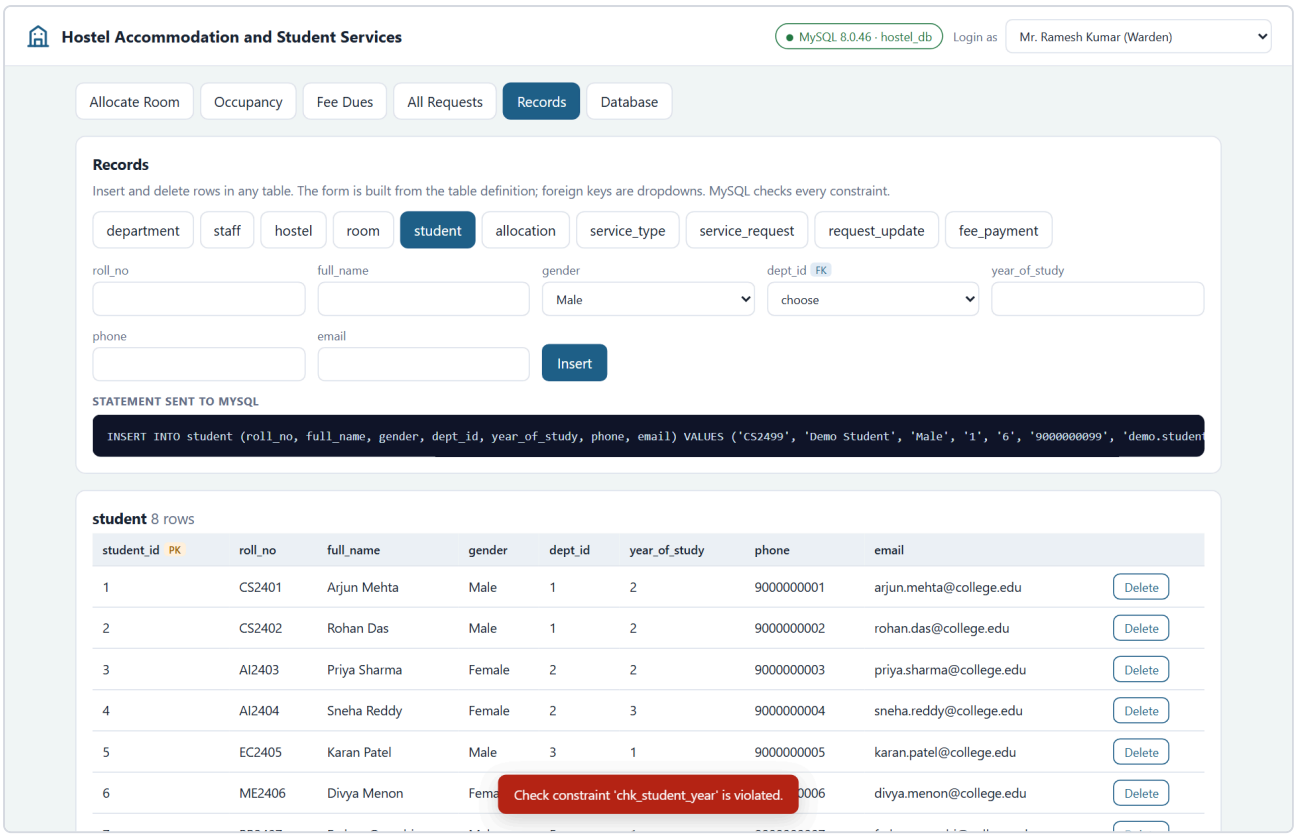
RULE Figure 30. Deleting a hostel in use is refused

Aryabhata Block still has rooms, so MySQL rejects the delete with a foreign key error and nothing changes.



RULE **Figure 31. An invalid value is rejected**

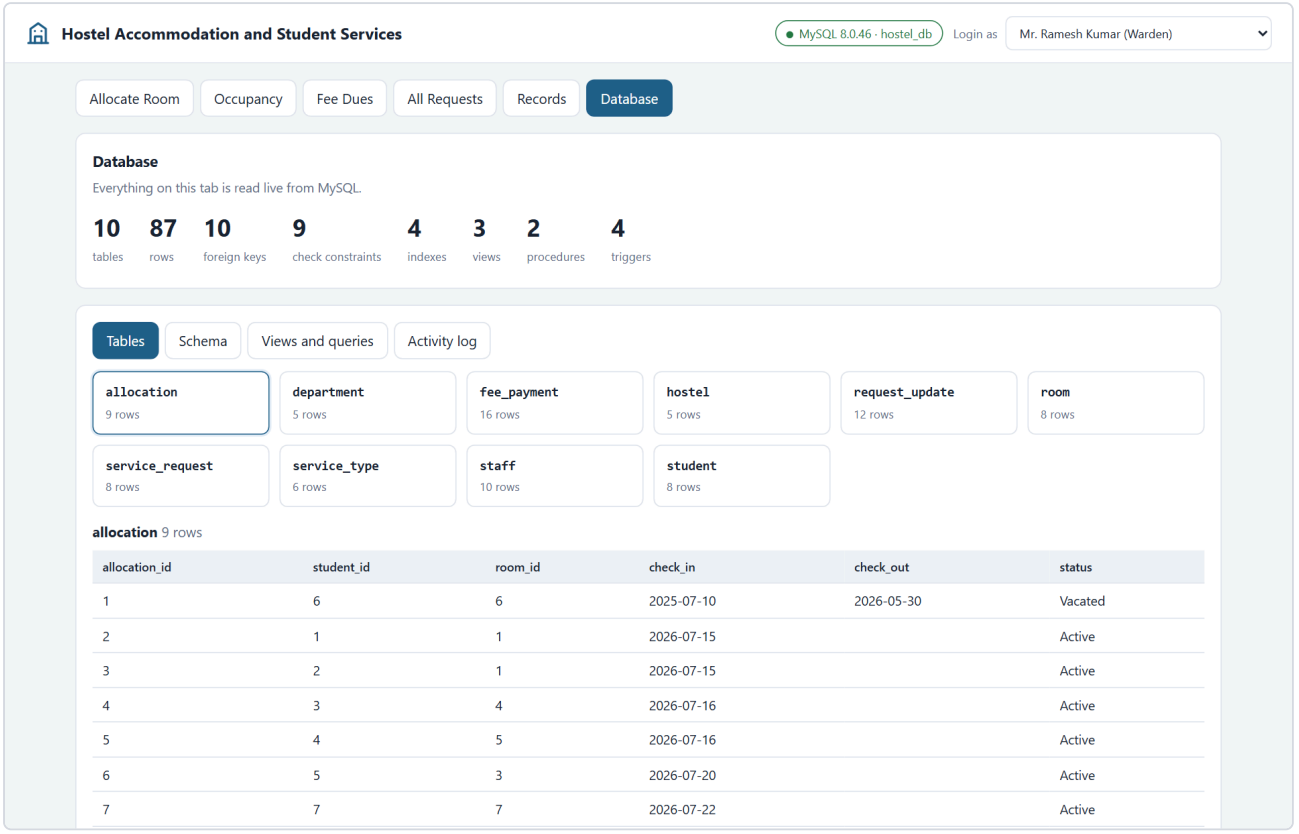
Year of study 6 breaks the CHECK constraint chk_student_year (1 to 4), so MySQL refuses the insert.



8. Database tab: what is stored in MySQL

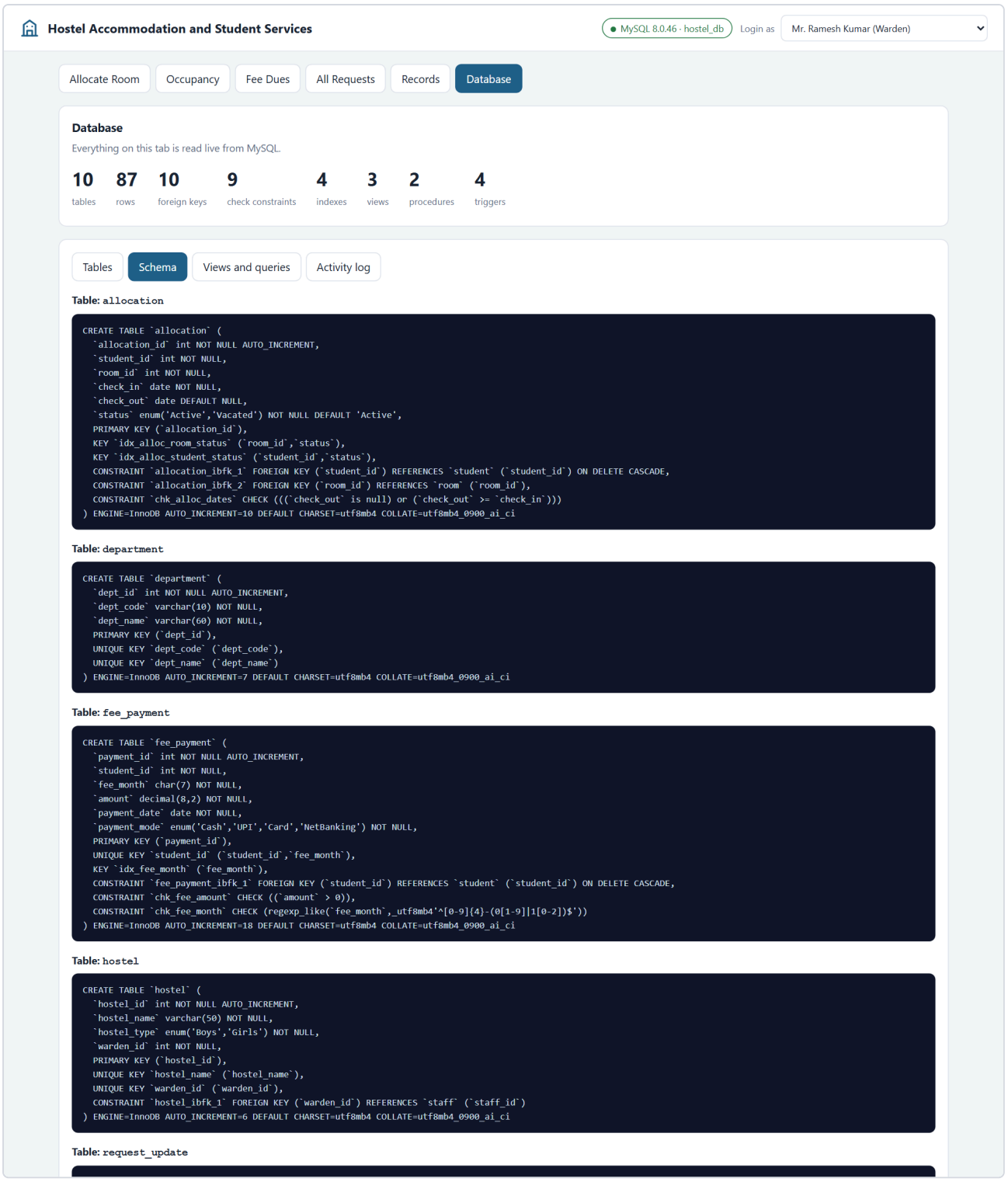
VIEW **Figure 32. Tables**

Row counts for all ten tables and the number of keys, checks, indexes, views, procedures and triggers, read live from MySQL.



VIEW **Figure 33. Schema (part 1 of 4)**

The real CREATE TABLE, view, procedure and trigger definitions from MySQL.



VIEW Figure 33. Schema (part 2 of 4)

```
CREATE TABLE `request_update` (
  `update_id` int NOT NULL AUTO_INCREMENT,
  `request_id` int NOT NULL,
  `staff_id` int NOT NULL,
  `new_status` enum('Assigned','In Progress','Resolved') NOT NULL,
  `remarks` varchar(200) DEFAULT NULL,
  `updated_on` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`update_id`),
  UNIQUE KEY `request_id` (`request_id`,`new_status`),
  KEY `staff_id` (`staff_id`),
  CONSTRAINT `request_update_ibfk_1` FOREIGN KEY (`request_id`) REFERENCES `service_request` (`request_id`) ON DELETE CASCADE,
  CONSTRAINT `request_update_ibfk_2` FOREIGN KEY (`staff_id`) REFERENCES `staff` (`staff_id`)
) ENGINE=InnoDB AUTO_INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Table: `room`

```
CREATE TABLE `room` (
  `room_id` int NOT NULL AUTO_INCREMENT,
  `hostel_id` int NOT NULL,
  `room_no` varchar(10) NOT NULL,
  `room_type` enum('Single','Double','Triple') NOT NULL,
  `capacity` tinyint NOT NULL,
  `monthly_rent` decimal(8,2) NOT NULL,
  PRIMARY KEY (`room_id`),
  UNIQUE KEY `hostel_id` (`hostel_id`,`room_no`),
  CONSTRAINT `room_ibfk_1` FOREIGN KEY (`hostel_id`) REFERENCES `hostel` (`hostel_id`),
  CONSTRAINT `chk_room_capacity` CHECK ((`capacity` between 1 and 3)),
  CONSTRAINT `chk_room_rent` CHECK ((`monthly_rent` > 0))
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Table: `service_request`

```
CREATE TABLE `service_request` (
  `request_id` int NOT NULL AUTO_INCREMENT,
  `student_id` int NOT NULL,
  `service_type_id` int NOT NULL,
  `description` varchar(200) NOT NULL,
  `request_date` date NOT NULL,
  `status` enum('Open','Assigned','In Progress','Resolved','Cancelled') NOT NULL DEFAULT 'Open',
  `resolved_on` date DEFAULT NULL,
  PRIMARY KEY (`request_id`),
  KEY `student_id` (`student_id`),
  KEY `service_type_id` (`service_type_id`),
  KEY `idx_request_status` (`status`),
  CONSTRAINT `service_request_ibfk_1` FOREIGN KEY (`student_id`) REFERENCES `student` (`student_id`) ON DELETE CASCADE,
  CONSTRAINT `service_request_ibfk_2` FOREIGN KEY (`service_type_id`) REFERENCES `service_type` (`service_type_id`),
  CONSTRAINT `chk_resolved_date` CHECK (((`resolved_on` is null) or (`resolved_on` >= `request_date`)))
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Table: `service_type`

```
CREATE TABLE `service_type` (
  `service_type_id` int NOT NULL AUTO_INCREMENT,
  `type_name` varchar(30) NOT NULL,
  `target_days` tinyint NOT NULL,
  PRIMARY KEY (`service_type_id`),
  UNIQUE KEY `type_name` (`type_name`),
  CONSTRAINT `chk_target_days` CHECK ((`target_days` > 0))
) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Table: `staff`

```
CREATE TABLE `staff` (
  `staff_id` int NOT NULL AUTO_INCREMENT,
  `full_name` varchar(60) NOT NULL,
  `designation` enum('Warden','Electrician','Plumber','Housekeeping','Network Technician','Mess Supervisor') NOT NULL,
  `phone` char(10) NOT NULL,
  PRIMARY KEY (`staff_id`),
  UNIQUE KEY `phone` (`phone`),
  CONSTRAINT `chk_staff_phone` CHECK (regexp_like(`phone`,_latin1'^[6-9][0-9]{9}$'))
) ENGINE=InnoDB AUTO_INCREMENT=11 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Table: `student`

```
CREATE TABLE `student` (
  `student_id` int NOT NULL AUTO_INCREMENT,
  `roll_no` varchar(15) NOT NULL,
  `full_name` varchar(60) NOT NULL,
  `gender` enum('Male','Female') NOT NULL,
  `dept_id` int NOT NULL,
```

VIEW Figure 33. Schema (part 3 of 4)

```
  `year_of_study` tinyint NOT NULL,
  `phone` char(10) NOT NULL,
  `email` varchar(60) NOT NULL,
  PRIMARY KEY (`student_id`),
  UNIQUE KEY `roll_no` (`roll_no`),
  UNIQUE KEY `email` (`email`),
  KEY `dept_id` (`dept_id`),
  CONSTRAINT `student_ibfk_1` FOREIGN KEY (`dept_id`) REFERENCES `department` (`dept_id`),
  CONSTRAINT `chk_student_year` CHECK ((`year_of_study` between 1 and 4))
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

View: `v_fee_summary`

```
CREATE VIEW `v_fee_summary` AS select `s`.`student_id` AS `student_id`,`s`.`roll_no` AS `roll_no`,`s`.`full_name` AS `full_name`,`h`.`hostel_name` AS `hostel_name`,`
```

View: `v_open_requests`

```
CREATE VIEW `v_open_requests` AS select `sr`.`request_id` AS `request_id`,`s`.`roll_no` AS `roll_no`,`s`.`full_name` AS `full_name`,`h`.`hostel_name` AS `hostel_name`
```

View: `v_room_occupancy`

```
CREATE VIEW `v_room_occupancy` AS select `r`.`room_id` AS `room_id`,`h`.`hostel_name` AS `hostel_name`,`h`.`hostel_type` AS `hostel_type`,`r`.`room_no` AS `room_no`,`
```

Procedure: `sp_allocate_room`

```
CREATE PROCEDURE `sp_allocate_room`(
    IN p_student_id INT, IN p_room_id INT, IN p_check_in DATE)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM student WHERE student_id = p_student_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student does not exist';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM room WHERE room_id = p_room_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room does not exist';
    END IF;
    INSERT INTO allocation(student_id, room_id, check_in)
    VALUES (p_student_id, p_room_id, p_check_in);
END
```

Procedure: `sp_raise_request`

```
CREATE PROCEDURE `sp_raise_request`(
    IN p_student_id INT, IN p_service_type_id INT,
    IN p_description VARCHAR(200), IN p_request_date DATE)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM allocation
                   WHERE student_id = p_student_id AND status = 'Active') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a current hostel resident can raise a request';
    END IF;
    IF EXISTS (SELECT 1 FROM service_request
               WHERE student_id = p_student_id
                 AND service_type_id = p_service_type_id
                 AND status IN ('Open','Assigned','In Progress')) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'An open request of this type already exists';
    END IF;
    INSERT INTO service_request(student_id, service_type_id, description, request_date)
    VALUES (p_student_id, p_service_type_id, p_description, p_request_date);
END
```

Trigger: `trg_allocation_before_insert`

```
CREATE TRIGGER `trg_allocation_before_insert` BEFORE INSERT ON `allocation` FOR EACH ROW BEGIN
    DECLARE v_capacity TINYINT;
    DECLARE v_occupied INT;
    DECLARE v_hostel_type VARCHAR(5);
    DECLARE v_gender VARCHAR(6);

    IF NEW.status = 'Active' THEN
        IF EXISTS (SELECT 1 FROM allocation
                   WHERE student_id = NEW.student_id AND status = 'Active') THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student already has an active room';
        END IF;

        SELECT r.capacity, h.hostel_type INTO v_capacity, v_hostel_type
        FROM room r JOIN hostel h ON h.hostel_id = r.hostel_id
        WHERE r.room_id = NEW.room_id;

        SELECT gender INTO v_gender FROM student WHERE student_id = NEW.student_id;
        IF (v_hostel_type = 'Boys' AND v_gender <> 'Male')
           OR (v_hostel_type = 'Girls' AND v_gender <> 'Female') THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Hostel type does not match student gender';
```

VIEW Figure 33. Schema (part 4 of 4)

```
        END IF;

        SELECT COUNT(*) INTO v_occupied FROM allocation
        WHERE room_id = NEW.room_id AND status = 'Active';
        IF v_occupied >= v_capacity THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is full';
        END IF;
    END IF;
END
```

Trigger: `trg_update_before_insert`

```
CREATE TRIGGER `trg_update_before_insert` BEFORE INSERT ON `request_update` FOR EACH ROW BEGIN
    DECLARE v_status VARCHAR(15);
    SELECT status INTO v_status FROM service_request WHERE request_id = NEW.request_id;

    IF v_status IN ('Resolved','Cancelled') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Service request is already closed';
    END IF;
    IF NEW.new_status = 'Assigned' AND v_status <> 'Open' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request is already assigned';
    END IF;
    IF NEW.new_status = 'In Progress' AND v_status <> 'Assigned' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request must be assigned before work starts';
    END IF;
    IF NEW.new_status = 'Resolved' AND v_status <> 'In Progress' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request must be In Progress before it is resolved';
    END IF;
END
```

Trigger: `trg_update_after_insert`

```
CREATE TRIGGER `trg_update_after_insert` AFTER INSERT ON `request_update` FOR EACH ROW BEGIN
    IF NEW.new_status = 'Resolved' THEN
        UPDATE service_request
        SET status = 'Resolved', resolved_on = DATE(NEW.updated_on)
        WHERE request_id = NEW.request_id;
    ELSE
        UPDATE service_request SET status = NEW.new_status
        WHERE request_id = NEW.request_id;
    END IF;
END
```

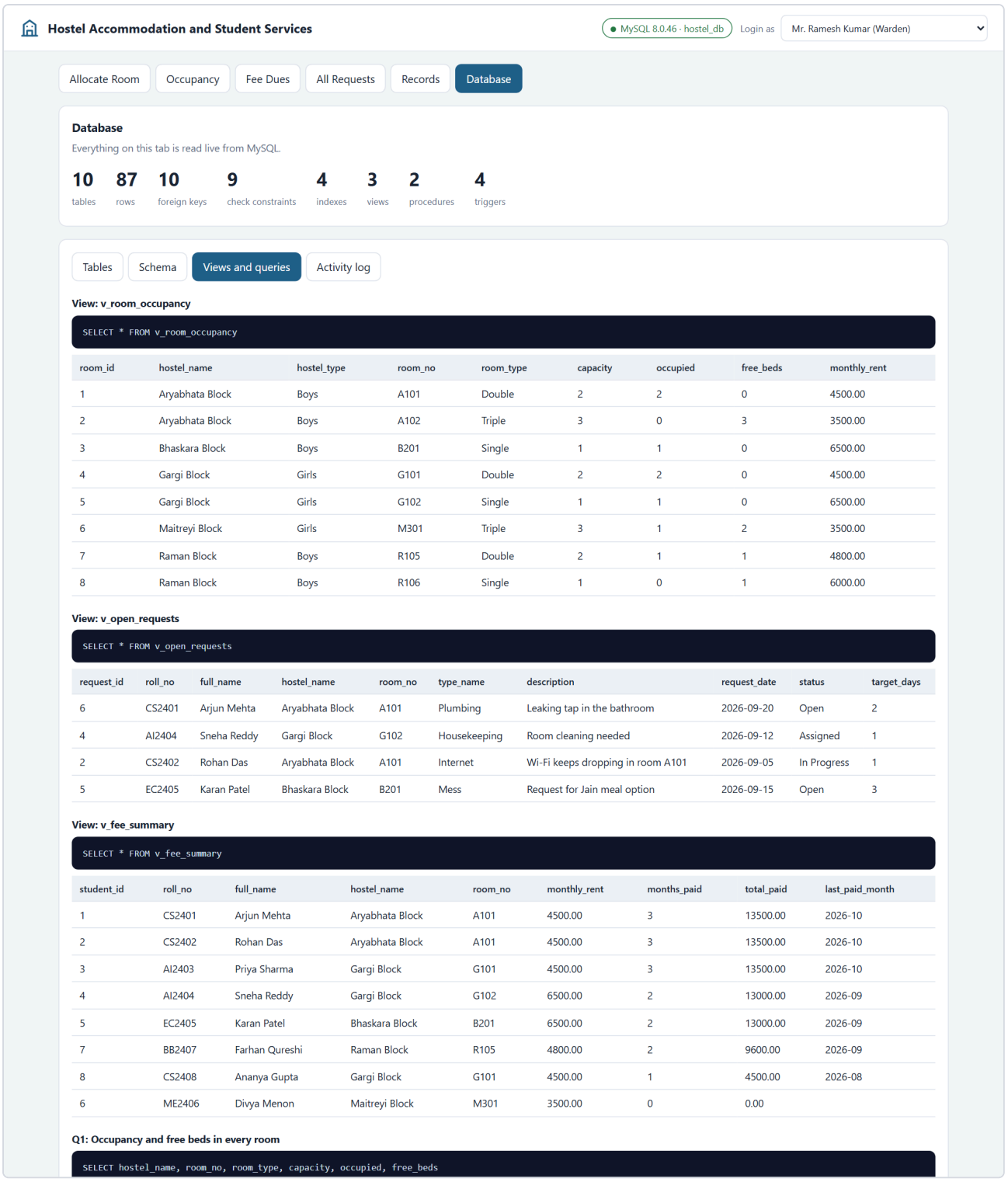
Trigger: `trg_fee_before_insert`

```
CREATE TRIGGER `trg_fee_before_insert` BEFORE INSERT ON `fee_payment` FOR EACH ROW BEGIN
    DECLARE v_rent DECIMAL(8,2);
    SELECT r.monthly_rent INTO v_rent
    FROM allocation a JOIN room r ON r.room_id = a.room_id
    WHERE a.student_id = NEW.student_id AND a.status = 'Active';

    IF v_rent IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student has no active room';
    END IF;
    IF NEW.amount <> v_rent THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Amount must equal the monthly rent of the room';
    END IF;
END
```

VIEW **Figure 34. Views and report queries (part 1 of 4)**

The three views and the nine report queries Q1 to Q9 with their live results.



VIEW Figure 34. Views and report queries (part 2 of 4)

```
FROM v_room_occupancy
ORDER BY hostel_name, room_no
```

hostel_name	room_no	room_type	capacity	occupied	free_beds
Aryabhata Block	A101	Double	2	2	0
Aryabhata Block	A102	Triple	3	0	3
Bhaskara Block	B201	Single	1	1	0
Gargi Block	G101	Double	2	2	0
Gargi Block	G102	Single	1	1	0
Maitreyi Block	M301	Triple	3	1	2
Raman Block	R105	Double	2	1	1
Raman Block	R106	Single	1	0	1

Q2: Rooms with a free bed in girls' hostels

```
SELECT hostel_name, room_no, room_type, free_beds, monthly_rent
FROM v_room_occupancy
WHERE hostel_type = 'Girls' AND free_beds > 0
ORDER BY monthly_rent
```

hostel_name	room_no	room_type	free_beds	monthly_rent
Maitreyi Block	M301	Triple	2	3500.00

Q3: Who stays where

```
SELECT s.roll_no, s.full_name, d.dept_code, h.hostel_name, r.room_no, a.check_in
FROM allocation a
JOIN student s    ON s.student_id = a.student_id
JOIN department d ON d.dept_id = s.dept_id
JOIN room r       ON r.room_id = a.room_id
JOIN hostel h     ON h.hostel_id = r.hostel_id
WHERE a.status = 'Active'
ORDER BY h.hostel_name, r.room_no
```

roll_no	full_name	dept_code	hostel_name	room_no	check_in
CS2401	Arjun Mehta	CSE	Aryabhata Block	A101	2026-07-15
CS2402	Rohan Das	CSE	Aryabhata Block	A101	2026-07-15
EC2405	Karan Patel	ECE	Bhaskara Block	B201	2026-07-20
AI2403	Priya Sharma	AIML	Gargi Block	G101	2026-07-16
CS2408	Ananya Gupta	CSE	Gargi Block	G101	2026-08-01
AI2404	Sneha Reddy	AIML	Gargi Block	G102	2026-07-16
ME2406	Divya Menon	MECH	Maitreyi Block	M301	2026-10-14
BB2407	Farhan Qureshi	BBA	Raman Block	R105	2026-07-22

Q4: Open service requests

```
SELECT request_id, full_name, hostel_name, room_no, type_name, request_date, status
FROM v_open_requests
ORDER BY request_date
```

request_id	full_name	hostel_name	room_no	type_name	request_date	status
2	Rohan Das	Aryabhata Block	A101	Internet	2026-09-05	In Progress
4	Sneha Reddy	Gargi Block	G102	Housekeeping	2026-09-12	Assigned
5	Karan Patel	Bhaskara Block	B201	Mess	2026-09-15	Open
6	Arjun Mehta	Aryabhata Block	A101	Plumbing	2026-09-20	Open

Q5: History of every request

```
SELECT sr.request_id, s.full_name, st.type_name, ru.new_status,
       f.full_name AS staff, ru.remarks, ru.updated_on
```

VIEW Figure 34. Views and report queries (part 3 of 4)

```
FROM service_request sr
JOIN student s        ON s.student_id = sr.student_id
JOIN service_type st ON st.service_type_id = sr.service_type_id
JOIN request_update ru ON ru.request_id = sr.request_id
JOIN staff f          ON f.staff_id = ru.staff_id
ORDER BY sr.request_id, ru.updated_on
```

request_id	full_name	type_name	new_status	staff	remarks	updated_on
1	Arjun Mehta	Electrical	Assigned	Mr. Prakash Yadav	Electrician assigned	2026-09-01 10:30:00
1	Arjun Mehta	Electrical	In Progress	Mr. Prakash Yadav	Checking fan capacitor	2026-09-01 16:00:00
1	Arjun Mehta	Electrical	Resolved	Mr. Prakash Yadav	Capacitor replaced, fan works	2026-09-02 11:15:00
2	Rohan Das	Internet	Assigned	Mr. Naveen Reddy	Network team informed	2026-09-05 12:00:00
2	Rohan Das	Internet	In Progress	Mr. Naveen Reddy	Access point being replaced	2026-09-06 09:45:00
3	Priya Sharma	Laundry	Assigned	Ms. Rekha Bai	Laundry vendor contacted	2026-09-08 14:00:00
3	Priya Sharma	Laundry	In Progress	Ms. Rekha Bai	Pickup rescheduled	2026-09-09 08:30:00
3	Priya Sharma	Laundry	Resolved	Ms. Rekha Bai	Clothes collected and returned	2026-09-10 18:00:00
4	Sneha Reddy	Housekeeping	Assigned	Ms. Rekha Bai	Housekeeping slot booked	2026-09-12 15:20:00
7	Farhan Qureshi	Electrical	Assigned	Mr. Prakash Yadav	Will check this evening	2026-10-14 23:27:57
7	Farhan Qureshi	Electrical	In Progress	Mr. Prakash Yadav	Starter of the tube light is faulty	2026-10-14 23:27:58
7	Farhan Qureshi	Electrical	Resolved	Mr. Prakash Yadav	Starter and tube replaced	2026-10-14 23:27:59

Q6: Residents who have not paid rent for October 2026

```
SELECT s.roll_no, s.full_name, h.hostel_name, r.room_no, r.monthly_rent AS due
FROM allocation a
JOIN student s ON s.student_id = a.student_id
JOIN room r    ON r.room_id = a.room_id
JOIN hostel h  ON h.hostel_id = r.hostel_id
WHERE a.status = 'Active'
  AND NOT EXISTS (SELECT 1 FROM fee_payment f
                  WHERE f.student_id = a.student_id AND f.fee_month = '2026-10')
```

roll_no	full_name	hostel_name	room_no	due
AI2404	Sneha Reddy	Gargi Block	G102	6500.00
EC2405	Karan Patel	Bhaskara Block	B201	6500.00
BB2407	Farhan Qureshi	Raman Block	R105	4800.00
CS2408	Ananya Gupta	Gargi Block	G101	4500.00
ME2406	Divya Menon	Maitreyi Block	M301	3500.00

Q7: Rent collected per hostel per month

```
SELECT h.hostel_name, f.fee_month, COUNT(*) AS payments, SUM(f.amount) AS collected
FROM fee_payment f
JOIN allocation a ON a.student_id = f.student_id AND a.status = 'Active'
JOIN room r       ON r.room_id = a.room_id
JOIN hostel h     ON h.hostel_id = r.hostel_id
GROUP BY h.hostel_name, f.fee_month
ORDER BY f.fee_month, h.hostel_name
```

hostel_name	fee_month	payments	collected
Aryabhata Block	2026-08	2	9000.00
Bhaskara Block	2026-08	1	6500.00
Gargi Block	2026-08	3	15500.00
Raman Block	2026-08	1	4800.00
Aryabhata Block	2026-09	2	9000.00
Bhaskara Block	2026-09	1	6500.00
Gargi Block	2026-09	2	11000.00
Raman Block	2026-09	1	4800.00

VIEW **Figure 34. Views and report queries (part 4 of 4)**

Aryabhata Block	2026-10	2	9000.00
Gargi Block	2026-10	1	4500.00

Q8: Resolved requests per service type against the target

```
SELECT st.type_name, st.target_days, COUNT(*) AS resolved,
       AVG(DATEDIFF(sr.resolved_on, sr.request_date)) AS avg_days
FROM service_request sr
JOIN service_type st ON st.service_type_id = sr.service_type_id
WHERE sr.status = 'Resolved'
GROUP BY st.service_type_id, st.type_name, st.target_days
```

type_name	target_days	resolved	avg_days
Electrical	2	2	6.0000
Laundry	2	1	2.0000

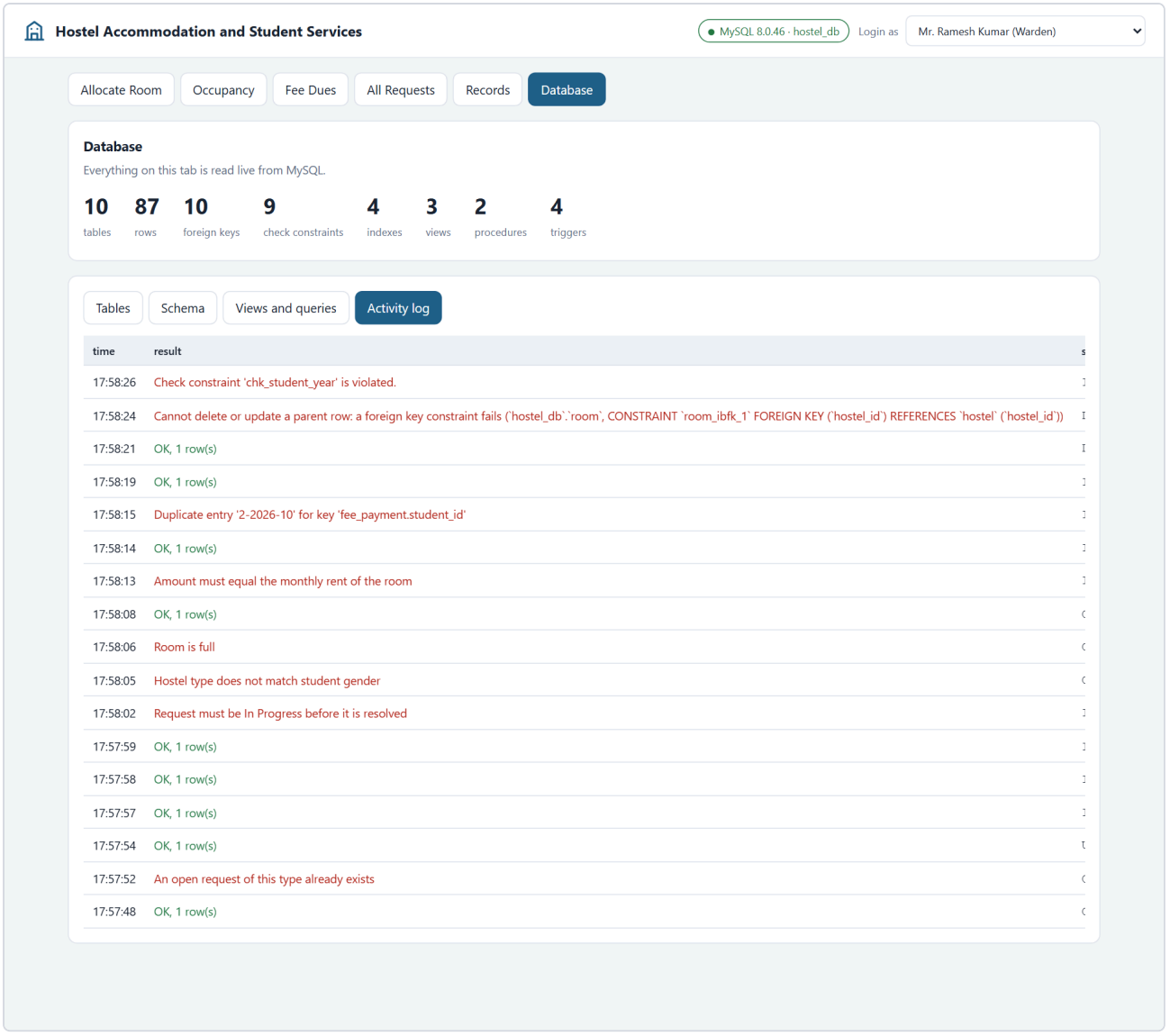
Q9: Hostel residents per department (at least two)

```
SELECT d.dept_name, COUNT(*) AS residents
FROM allocation a
JOIN student s    ON s.student_id = a.student_id
JOIN department d ON d.dept_id = s.dept_id
WHERE a.status = 'Active'
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(*) >= 2
```

dept_name	residents
Computer Science and Engineering	3
Artificial Intelligence and Machine Learning	2

VIEW **Figure 35. Activity log**

Every write sent to MySQL by the UI in this walkthrough, newest first, including the ones MySQL refused.



VIEW **Figure 36. Mobile layout, dark mode**
The same app on a phone-sized screen.

